

Arborescence du futur depot

Depot cible independant du projet medical. Les chemins ci-dessous sont ceux a creer dans un nouveau depot **b2b-commerce-mvp**. Ce PDF est une specification executable: chaque fichier marque FICHER COMPLET est a copier tel quel.

```
b2b-commerce-mvp/
|-- pom.xml
|-- .env.example
|-- docker-compose.yml
|-- README.md
|-- scripts/{dev-up.ps1, dev-down.ps1, verify.ps1, kind-up.ps1}
|-- infra/
|   |-- keycloak/realm-b2b.json
|   |-- postgres/init.sql
|   |-- k8s/{namespace.yaml, configmap.yaml, secrets.example.yaml, keycloak.yaml, kafka.yaml, postgres.yaml, gateway.yaml, services.yaml, web.yaml, ingress.yaml}
|-- gateway/{pom.xml, Dockerfile, src/main/java/com/acme/gateway/GatewayApplication.java, src/main/resources/application.yaml}
|-- tenant-service/{pom.xml, Dockerfile, src/main/java/com/acme/tenant/{... , src/main/resources/{application.yaml, db/migration/V1__tenant.sql}, src/test/{...}}
|-- catalog-service/{pom.xml, Dockerfile, src/main/java/com/acme/catalog/{CatalogApplication.java, domain, application, infrastructure, web, security}/{... , src/main/resources/{application.yaml, db/migration/V1__catalog.sql}, src/test/{...}}
|-- inventory-service/{pom.xml, Dockerfile, src/main/java/com/acme/inventory/{... , src/main/resources/{application.yaml, db/migration/V1__inventory.sql}, src/test/{...}}
|-- order-service/{pom.xml, Dockerfile, src/main/java/com/acme/order/{... , src/main/resources/{application.yaml, db/migration/V1__orders.sql}, src/test/{...}}
|-- web/{package.json, angular.json, tsconfig.json, src/{main.ts, index.html, styles.css, app/{...}}, Dockerfile, nginx.conf}
|-- mobile/{pubspec.yaml, lib/{main.dart, config.dart, auth_service.dart, api_client.dart, models.dart, product_page.dart, order_page.dart}, test/widget_test.dart}
```

Prerequis et commandes exactes

```
# Versions de reference
java --version           # openjdk 25 LTS
mvn --version            # Maven 3.9.11+
node --version           # Node 24 LTS
npm --version            # npm 11+
docker version          # Docker Engine/Desktop recent
kubectl version --client # kubectl compatible cluster
kind version            # kind recent
flutter --version        # Flutter 3.47 / Dart compatible

# Creer le nouveau depot (PowerShell)
mkdir b2b-commerce-mvp; Set-Location b2b-commerce-mvp
git init
git branch -M main
Copy-Item .env.example .env
docker compose config
docker compose up -d --build
mvn -T 1C clean verify
npm --prefix web ci
npm --prefix web run test -- --watch=false
npm --prefix web run build
flutter pub get --directory mobile
flutter test mobile

# Verification locale
Invoke-RestMethod http://localhost:8080/actuator/health
Start-Process http://localhost:8081/realms/b2b/account
# Kubernetes local
kind create cluster --name b2b-mvp
kubectl apply -f infra/k8s/namespace.yaml
kubectl apply -f infra/k8s/ -n b2b
kubectl -n b2b rollout status deployment/gateway --timeout=180s
kubectl -n b2b port-forward svc/gateway 8080:8080
```

Aucun secret reel n'est fourni. Les mots de passe de demonstration sont strictement locaux et doivent etre remplaces par Secret Manager/External Secrets en environnement partage.

FICHIER COMPLET

pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:
schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion><groupId>com.acme.b2b</groupId><artifactId>b2b-commerce</artifactId><version>0.
1.0-SNAPSHOT</version><packaging>pom</packaging>
  <modules><module>gateway</module><module>tenant-service</module><module>catalog-service</
module><module>inventory-service</module><module>order-service</module></modules>
  <parent><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-parent</artifactId><version>4.
1.1</version><relativePath/></parent>
  <properties><java.version>25</java.version><maven.compiler.release>25</maven.compiler.release><testcontainers.
version>2.0.2</testcontainers.version></properties>
  <dependencyManagement><dependencies><dependency><groupId>org.testcontainers</groupId><artifactId>testcontainers-
bom</artifactId><version>${testcontainers.version}</version><type>pom</type><scope>import</scope></dependency></
dependencies></dependencyManagement>
  <build><pluginManagement><plugins><plugin><groupId>org.apache.maven.plugins</groupId><artifactId>maven-surefire-
plugin</artifactId><version>3.5.4</version><configuration><useModulePath>false</useModulePath></configuration></
plugin></plugins></pluginManagement></build>
</project>
```

FICHIER COMPLET

gateway/pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"><modelVersion>4.0.0</modelVersion><parent><groupId>com.acme.
b2b</groupId><artifactId>b2b-commerce</artifactId><version>0.1.0-SNAPSHOT</version></parent><artifactId>gateway</
artifactId>
<dependencies><dependency><groupId>org.springframework.cloud</groupId><artifactId>spring-cloud-starter-gateway-
server-webflux</artifactId></dependency><dependency><groupId>org.springframework.boot</groupId><artifactId>spring-
boot-starter-oauth2-resource-server</artifactId></dependency><dependency><groupId>org.springframework.boot</
groupId><artifactId>spring-boot-starter-actuator</artifactId></dependency><dependency><groupId>org.springframework.
boot</groupId><artifactId>spring-boot-starter-validation</artifactId></dependency><dependency><groupId>org.
springframework.boot</groupId><artifactId>spring-boot-starter-test</artifactId><scope>test</scope></
dependency><dependency><groupId>org.springframework.security</groupId><artifactId>spring-security-test</
artifactId><scope>test</scope></dependency><dependency><groupId>org.testcontainers</
groupId><artifactId>postgresql</artifactId><scope>test</scope></dependency></dependencies>
<build><plugins><plugin><groupId>org.springframework.boot</groupId><artifactId>spring-boot-maven-plugin</
artifactId></plugin></plugins></build></project>
```

tenant-service/pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"><modelVersion>4.0.0</modelVersion><parent><groupId>com.acme.
b2b</groupId><artifactId>b2b-commerce</artifactId><version>0.1.0-SNAPSHOT</version></parent><artifactId>tenant-
service</artifactId>
<dependencies><dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-web</
artifactId></dependency><dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-
data-jpa</artifactId></dependency><dependency><groupId>org.flywaydb</groupId><artifactId>flyway-core</artifactId></
dependency><dependency><groupId>org.flywaydb</groupId><artifactId>flyway-database-postgresql</artifactId></
dependency><dependency><groupId>org.postgresql</groupId><artifactId>postgresql</artifactId><scope>runtime</scope></
dependency><dependency><groupId>org.springframework.kafka</groupId><artifactId>spring-kafka</artifactId></
dependency><dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-oauth2-resource-
server</artifactId></dependency><dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-
starter-actuator</artifactId></dependency><dependency><groupId>org.springframework.boot</
groupId><artifactId>spring-boot-starter-validation</artifactId></dependency><dependency><groupId>org.
springframework.boot</groupId><artifactId>spring-boot-starter-test</artifactId><scope>test</scope></
dependency><dependency><groupId>org.springframework.security</groupId><artifactId>spring-security-test</
artifactId><scope>test</scope></dependency><dependency><groupId>org.testcontainers</
groupId><artifactId>postgresql</artifactId><scope>test</scope></dependency></dependencies>
<build><plugins><plugin><groupId>org.springframework.boot</groupId><artifactId>spring-boot-maven-plugin</
artifactId></plugin></plugins></build></project>
```

FICHIER COMPLET

catalog-service/pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"><modelVersion>4.0.0</modelVersion><parent><groupId>com.acme.
b2b</groupId><artifactId>b2b-commerce</artifactId><version>0.1.0-SNAPSHOT</version></parent><artifactId>catalog-
service</artifactId>
<dependencies><dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-web</
artifactId></dependency><dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-
data-jpa</artifactId></dependency><dependency><groupId>org.flywaydb</groupId><artifactId>flyway-core</artifactId></
dependency><dependency><groupId>org.flywaydb</groupId><artifactId>flyway-database-postgresql</artifactId></
dependency><dependency><groupId>org.postgresql</groupId><artifactId>postgresql</artifactId><scope>runtime</scope></
dependency><dependency><groupId>org.springframework.kafka</groupId><artifactId>spring-kafka</artifactId></
dependency><dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-oauth2-resource-
server</artifactId></dependency><dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-
starter-actuator</artifactId></dependency><dependency><groupId>org.springframework.boot</
groupId><artifactId>spring-boot-starter-validation</artifactId></dependency><dependency><groupId>org.
springframework.boot</groupId><artifactId>spring-boot-starter-test</artifactId><scope>test</scope></
dependency><dependency><groupId>org.springframework.security</groupId><artifactId>spring-security-test</
artifactId><scope>test</scope></dependency><dependency><groupId>org.testcontainers</
groupId><artifactId>postgresql</artifactId><scope>test</scope></dependency></dependencies>
<build><plugins><plugin><groupId>org.springframework.boot</groupId><artifactId>spring-boot-maven-plugin</
artifactId></plugin></plugins></build></project>
```

inventory-service/pom.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"><modelVersion>4.0.0</modelVersion><parent><groupId>com.acme.
b2b</groupId><artifactId>b2b-commerce</artifactId><version>0.1.0-SNAPSHOT</version></parent><artifactId>inventory-
service</artifactId>
<dependencies><dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-web</
artifactId></dependency><dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-
data-jpa</artifactId></dependency><dependency><groupId>org.flywaydb</groupId><artifactId>flyway-core</artifactId></
dependency><dependency><groupId>org.flywaydb</groupId><artifactId>flyway-database-postgresql</artifactId></
dependency><dependency><groupId>org.postgresql</groupId><artifactId>postgresql</artifactId><scope>runtime</scope></
dependency><dependency><groupId>org.springframework.kafka</groupId><artifactId>spring-kafka</artifactId></
dependency><dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-oauth2-resource-
server</artifactId></dependency><dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-
starter-actuator</artifactId></dependency><dependency><groupId>org.springframework.boot</
groupId><artifactId>spring-boot-starter-validation</artifactId></dependency><dependency><groupId>org.
springframework.boot</groupId><artifactId>spring-boot-starter-test</artifactId><scope>test</scope></
dependency><dependency><groupId>org.springframework.security</groupId><artifactId>spring-security-test</
artifactId><scope>test</scope></dependency><dependency><groupId>org.testcontainers</
groupId><artifactId>postgresql</artifactId><scope>test</scope></dependency></dependencies>
<build><plugins><plugin><groupId>org.springframework.boot</groupId><artifactId>spring-boot-maven-plugin</
artifactId></plugin></plugins></build></project>

```

FICHIER COMPLET

order-service/pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"><modelVersion>4.0.0</modelVersion><parent><groupId>com.acme.
b2b</groupId><artifactId>b2b-commerce</artifactId><version>0.1.0-SNAPSHOT</version></parent><artifactId>order-
service</artifactId>
<dependencies><dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-web</
artifactId></dependency><dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-
data-jpa</artifactId></dependency><dependency><groupId>org.flywaydb</groupId><artifactId>flyway-core</artifactId></
dependency><dependency><groupId>org.flywaydb</groupId><artifactId>flyway-database-postgresql</artifactId></
dependency><dependency><groupId>org.postgresql</groupId><artifactId>postgresql</artifactId><scope>runtime</scope></
dependency><dependency><groupId>org.springframework.kafka</groupId><artifactId>spring-kafka</artifactId></
dependency><dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-oauth2-resource-
server</artifactId></dependency><dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-
starter-actuator</artifactId></dependency><dependency><groupId>org.springframework.boot</
groupId><artifactId>spring-boot-starter-validation</artifactId></dependency><dependency><groupId>org.
springframework.boot</groupId><artifactId>spring-boot-starter-test</artifactId><scope>test</scope></
dependency><dependency><groupId>org.springframework.security</groupId><artifactId>spring-security-test</
artifactId><scope>test</scope></dependency><dependency><groupId>org.testcontainers</
groupId><artifactId>postgresql</artifactId><scope>test</scope></dependency></dependencies>
<build><plugins><plugin><groupId>org.springframework.boot</groupId><artifactId>spring-boot-maven-plugin</
artifactId></plugin></plugins></build></project>
```

FICHIER COMPLET

.env.example

```
POSTGRES_USER=b2b
POSTGRES_PASSWORD=local-only-change-me
KEYCLOAK_ADMIN=admin
KEYCLOAK_ADMIN_PASSWORD=local-only-change-me
DEMO_USER=buyer@demo.test
DEMO_PASSWORD=local-only-change-me
```

FICHIER COMPLET

infra/postgres/init.sql

```
CREATE DATABASE tenant_db;  
CREATE DATABASE catalog_db;  
CREATE DATABASE inventory_db;  
CREATE DATABASE order_db;  
GRANT ALL PRIVILEGES ON DATABASE tenant_db TO b2b;  
GRANT ALL PRIVILEGES ON DATABASE catalog_db TO b2b;  
GRANT ALL PRIVILEGES ON DATABASE inventory_db TO b2b;  
GRANT ALL PRIVILEGES ON DATABASE order_db TO b2b;
```

FICHER COMPLET

gateway/src/main/java/com/acme/gateway/GatewayApplication.java

```
package com.acme.gateway;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
@SpringBootApplication public class GatewayApplication { public static void main(String[] args){ SpringApplication.
run(GatewayApplication.class,args); } }
```

FICHIER COMPLET

gateway/src/main/java/com/acme/gateway/SecurityConfig.java

```
package com.acme.gateway;
import org.springframework.context.annotation.*; import org.springframework.security.config.web.server.
ServerHttpSecurity; import org.springframework.security.web.server.SecurityWebFilterChain;
@Configuration class SecurityConfig { @Bean SecurityWebFilterChain chain(ServerHttpSecurity http){ return http.
csrf(ServerHttpSecurity.CsrfSpec::disable).authorizeExchange(x->x.pathMatchers("/actuator/health").permitAll().
anyExchange().authenticated()).oauth2ResourceServer(o->o.jwt(j->{})).build(); } }
```

FICHER COMPLET

gateway/src/main/resources/application.yml

```
server:
  port: 8080
spring:
  application.name: gateway
  security.oauth2.resourceserver.jwt.issuer-uri: ${ISSUER_URI:http://keycloak:8080/realms/b2b}
  cloud.gateway.server.webflux.routes:
    - id: tenants
      uri: ${TENANT_URL:http://tenant-service:8080}
      predicates: [Path=/api/tenants/**]
    - id: catalog
      uri: ${CATALOG_URL:http://catalog-service:8080}
      predicates: [Path=/api/products/**]
    - id: inventory
      uri: ${INVENTORY_URL:http://inventory-service:8080}
      predicates: [Path=/api/stocks/**]
    - id: orders
      uri: ${ORDER_URL:http://order-service:8080}
      predicates: [Path=/api/orders/**]
  management.endpoints.web.exposure.include: health,info,prometheus
```

FICHER COMPLET

tenant-service/src/main/java/com/acme/tenant/TenantApplication.java

```
package com.acme.tenant;
import org.springframework.boot.SpringApplication; import org.springframework.boot.autoconfigure.
SpringBootApplication;
@SpringBootApplication public class TenantApplication { public static void main(String[] args){SpringApplication.
run(TenantApplication.class,args);} }
```

FICHER COMPLET

tenant-service/src/main/java/com/acme/tenant/security/TenantContext.java

```
package com.acme.tenant.security;
public final class TenantContext { private static final ThreadLocal<String> CURRENT=new ThreadLocal<>(); private
TenantContext(){} public static void set(String id){CURRENT.set(id);} public static String require(){String
id=CURRENT.get(); if(id==null||id.isBlank()) throw new IllegalStateException("tenant claim absent"); return id;}
public static void clear(){CURRENT.remove();} }
```

FICHIER COMPLET

tenant-service/src/main/java/com/acme/tenant/security/TenantFilter.java

```
package com.acme.tenant.security;
import jakarta.servlet.*; import jakarta.servlet.http.*; import java.io.IOException; import org.springframework.
security.oauth2.server.resource.authentication.JwtAuthenticationToken; import org.springframework.stereotype.
Component; import org.springframework.web.filter.OncePerRequestFilter;
@Component public class TenantFilter extends OncePerRequestFilter { protected void doFilterInternal(
HttpServletRequest r, HttpServletResponse s, FilterChain c) throws ServletException, IOException{ try{ if(r.
getUserPrincipal() instanceof JwtAuthenticationToken jwt) TenantContext.set(jwt.getToken().getClaimAsString(
"tenant_id")); c.doFilter(r,s); } finally{TenantContext.clear();} } }
```

FICHER COMPLET

tenant-service/src/main/java/com/acme/tenant/security/SecurityConfig.java

```
package com.acme.tenant.security;
import org.springframework.context.annotation.*; import org.springframework.security.config.annotation.web.
builders.HttpSecurity; import org.springframework.security.web.SecurityFilterChain; import org.springframework.
security.config.http.SessionCreationPolicy;
@Configuration public class SecurityConfig { @Bean SecurityFilterChain chain(HttpSecurity h)throws Exception{
return h.csrf(c->c.disable()).sessionManagement(s->s.sessionCreationPolicy(SessionCreationPolicy.STATELESS)).
authorizeHttpRequests(a->a.requestMatchers("/actuator/health").permitAll().anyRequest().authenticated()).
oauth2ResourceServer(o->o.jwt(j->{})).build();} }
```

FICHER COMPLET

tenant-service/src/main/resources/application.yml

```
server:
  port: 8080
spring:
  application.name: tenant-service
  datasource:
    url: ${DB_URL:jdbc:postgresql://postgres:5432/tenant_db}
    username: ${DB_USER:b2b}
    password: ${DB_PASSWORD:local-only-change-me}
  jpa:
    open-in-view: false
    hibernate.ddl-auto: validate
  flyway.enabled: true
  security.oauth2.resourceserver.jwt.issuer-uri: ${ISSUER_URI:http://keycloak:8080/realms/b2b}
  kafka.bootstrap.servers: ${KAFKA_BOOTSTRAP:kafka:9092}
management.endpoints.web.exposure.include: health,info,prometheus
```

FICHER COMPLET

catalog-service/src/main/java/com/acme/catalog/CatalogApplication.java

```
package com.acme.catalog;
import org.springframework.boot.SpringApplication; import org.springframework.boot.autoconfigure.
SpringBootApplication;
@SpringBootApplication public class CatalogApplication { public static void main(String[] args){SpringApplication.
run(CatalogApplication.class,args);} }
```

FICHIER COMPLET

catalog-service/src/main/java/com/acme/catalog/security/TenantContext.java

```
package com.acme.catalog.security;
public final class TenantContext { private static final ThreadLocal<String> CURRENT=new ThreadLocal<>(); private
TenantContext(){} public static void set(String id){CURRENT.set(id);} public static String require(){String
id=CURRENT.get(); if(id==null||id.isBlank()) throw new IllegalStateException("tenant claim absent"); return id;}
public static void clear(){CURRENT.remove();} }
```

FICHER COMPLET

catalog-service/src/main/java/com/acme/catalog/security/TenantFilter.java

```
package com.acme.catalog.security;
import jakarta.servlet.*; import jakarta.servlet.http.*; import java.io.IOException; import org.springframework.
security.oauth2.server.resource.authentication.JwtAuthenticationToken; import org.springframework.stereotype.
Component; import org.springframework.web.filter.OncePerRequestFilter;
@Component public class TenantFilter extends OncePerRequestFilter { protected void doFilterInternal(
HttpServletRequest r, HttpServletResponse s, FilterChain c) throws ServletException, IOException{ try{ if(r.
getUserPrincipal() instanceof JwtAuthenticationToken jwt) TenantContext.set(jwt.getToken().getClaimAsString(
"tenant_id")); c.doFilter(r,s); } finally{TenantContext.clear();} } }
```

FICHER COMPLET

catalog-service/src/main/java/com/acme/catalog/security/SecurityConfig.java

```
package com.acme.catalog.security;
import org.springframework.context.annotation.*; import org.springframework.security.config.annotation.web.
builders.HttpSecurity; import org.springframework.security.web.SecurityFilterChain; import org.springframework.
security.config.http.SessionCreationPolicy;
@Configuration public class SecurityConfig { @Bean SecurityFilterChain chain(HttpSecurity h)throws Exception{
return h.csrf(c->c.disable()).sessionManagement(s->s.sessionCreationPolicy(SessionCreationPolicy.STATELESS)).
authorizeHttpRequests(a->a.requestMatchers("/actuator/health").permitAll().anyRequest().authenticated()).
oauth2ResourceServer(o->o.jwt(j->{})).build();} }
```

FICHER COMPLET

catalog-service/src/main/resources/application.yml

```
server:
  port: 8080
spring:
  application.name: catalog-service
  datasource:
    url: ${DB_URL:jdbc:postgresql://postgres:5432/catalog_db}
    username: ${DB_USER:b2b}
    password: ${DB_PASSWORD:local-only-change-me}
  jpa:
    open-in-view: false
    hibernate.ddl-auto: validate
  flyway.enabled: true
  security.oauth2.resourceserver.jwt.issuer-uri: ${ISSUER_URI:http://keycloak:8080/realms/b2b}
  kafka.bootstrap.servers: ${KAFKA_BOOTSTRAP:kafka:9092}
management.endpoints.web.exposure.include: health,info,prometheus
```

FICHIER COMPLET

inventory-service/src/main/java/com/acme/inventory/InventoryApplication.java

```
package com.acme.inventory;
import org.springframework.boot.SpringApplication; import org.springframework.boot.autoconfigure.
SpringBootApplication;
@SpringBootApplication public class InventoryApplication { public static void main(String[] args){
SpringApplication.run(InventoryApplication.class,args);} }
```

FICHIER COMPLET

inventory-service/src/main/java/com/acme/inventory/security/TenantContext.java

```
package com.acme.inventory.security;
public final class TenantContext { private static final ThreadLocal<String> CURRENT=new ThreadLocal<>(); private
TenantContext(){} public static void set(String id){CURRENT.set(id);} public static String require(){String
id=CURRENT.get(); if(id==null||id.isBlank()) throw new IllegalStateException("tenant claim absent"); return id;}
public static void clear(){CURRENT.remove();} }
```

FICHIER COMPLET

inventory-service/src/main/java/com/acme/inventory/security/TenantFilter.java

```
package com.acme.inventory.security;
import jakarta.servlet.*; import jakarta.servlet.http.*; import java.io.IOException; import org.springframework.
security.oauth2.server.resource.authentication.JwtAuthenticationToken; import org.springframework.stereotype.
Component; import org.springframework.web.filter.OncePerRequestFilter;
@Component public class TenantFilter extends OncePerRequestFilter { protected void doFilterInternal(
HttpServletRequest r, HttpServletResponse s, FilterChain c) throws ServletException, IOException{ try{ if(r.
getUserPrincipal() instanceof JwtAuthenticationToken jwt) TenantContext.set(jwt.getToken().getClaimAsString(
"tenant_id")); c.doFilter(r,s); } finally{TenantContext.clear();} } }
```

FICHER COMPLET

inventory-service/src/main/java/com/acme/inventory/security/SecurityConfig.java

```
package com.acme.inventory.security;
import org.springframework.context.annotation.*; import org.springframework.security.config.annotation.web.
builders.HttpSecurity; import org.springframework.security.web.SecurityFilterChain; import org.springframework.
security.config.http.SessionCreationPolicy;
@Configuration public class SecurityConfig { @Bean SecurityFilterChain chain(HttpSecurity h)throws Exception{
return h.csrf(c->c.disable()).sessionManagement(s->s.sessionCreationPolicy(SessionCreationPolicy.STATELESS)).
authorizeHttpRequests(a->a.requestMatchers("/actuator/health").permitAll().anyRequest().authenticated()).
oauth2ResourceServer(o->o.jwt(j->{})).build();} }
```

FICHER COMPLET

inventory-service/src/main/resources/application.yml

```
server:
  port: 8080
spring:
  application.name: inventory-service
  datasource:
    url: ${DB_URL:jdbc:postgresql://postgres:5432/inventory_db}
    username: ${DB_USER:b2b}
    password: ${DB_PASSWORD:local-only-change-me}
  jpa:
    open-in-view: false
    hibernate.ddl-auto: validate
  flyway.enabled: true
  security.oauth2.resourceserver.jwt.issuer-uri: ${ISSUER_URI:http://keycloak:8080/realms/b2b}
  kafka.bootstrap.servers: ${KAFKA_BOOTSTRAP:kafka:9092}
management.endpoints.web.exposure.include: health,info,prometheus
```

FICHER COMPLET

order-service/src/main/java/com/acme/order/OrderApplication.java

```
package com.acme.order;
import org.springframework.boot.SpringApplication; import org.springframework.boot.autoconfigure.
SpringBootApplication;
@SpringBootApplication public class OrderApplication { public static void main(String[] args){SpringApplication.
run(OrderApplication.class,args);} }
```

FICHER COMPLET

order-service/src/main/java/com/acme/order/security/TenantContext.java

```
package com.acme.order.security;
public final class TenantContext { private static final ThreadLocal<String> CURRENT=new ThreadLocal<>(); private
TenantContext(){} public static void set(String id){CURRENT.set(id);} public static String require(){String
id=CURRENT.get(); if(id==null||id.isBlank()) throw new IllegalStateException("tenant claim absent"); return id;}
public static void clear(){CURRENT.remove();} }
```

FICHER COMPLET

order-service/src/main/java/com/acme/order/security/TenantFilter.java

```
package com.acme.order.security;
import jakarta.servlet.*; import jakarta.servlet.http.*; import java.io.IOException; import org.springframework.
security.oauth2.server.resource.authentication.JwtAuthenticationToken; import org.springframework.stereotype.
Component; import org.springframework.web.filter.OncePerRequestFilter;
@Component public class TenantFilter extends OncePerRequestFilter { protected void doFilterInternal(
HttpServletRequest r, HttpServletResponse s, FilterChain c) throws ServletException, IOException{ try{ if(r.
getUserPrincipal() instanceof JwtAuthenticationToken jwt) TenantContext.set(jwt.getToken().getClaimAsString(
"tenant_id")); c.doFilter(r,s); } finally{TenantContext.clear();} } }
```

FICHER COMPLET

order-service/src/main/java/com/acme/order/security/SecurityConfig.java

```
package com.acme.order.security;
import org.springframework.context.annotation.*; import org.springframework.security.config.annotation.web.
builders.HttpSecurity; import org.springframework.security.web.SecurityFilterChain; import org.springframework.
security.config.http.SessionCreationPolicy;
@Configuration public class SecurityConfig { @Bean SecurityFilterChain chain(HttpSecurity h)throws Exception{
return h.csrf(c->c.disable()).sessionManagement(s->s.sessionCreationPolicy(SessionCreationPolicy.STATELESS)).
authorizeHttpRequests(a->a.requestMatchers("/actuator/health").permitAll().anyRequest().authenticated()).
oauth2ResourceServer(o->o.jwt(j->{})).build();} }
```

FICHER COMPLET

order-service/src/main/resources/application.yml

```
server:
  port: 8080
spring:
  application.name: order-service
  datasource:
    url: ${DB_URL:jdbc:postgresql://postgres:5432/order_db}
    username: ${DB_USER:b2b}
    password: ${DB_PASSWORD:local-only-change-me}
  jpa:
    open-in-view: false
    hibernate.ddl-auto: validate
  flyway.enabled: true
  security.oauth2.resourceserver.jwt.issuer-uri: ${ISSUER_URI:http://keycloak:8080/realms/b2b}
  kafka.bootstrap.servers: ${KAFKA_BOOTSTRAP:kafka:9092}
management.endpoints.web.exposure.include: health,info,prometheus
```

FICHIER COMPLET

tenant-service/src/main/java/com/acme/tenant/domain/Tenant.java

```
package com.acme.tenant.domain;
import jakarta.persistence.*; import java.util.UUID;
@Entity @Table(name="tenants") public class Tenant { @Id private UUID id; @Column(nullable=false,unique=true)
private String slug; @Column(nullable=false) private String name; protected Tenant(){} public Tenant(UUID id,
String slug,String name){this.id=id;this.slug=slug;this.name=name;} public UUID getId(){return id;} public String
getSlug(){return slug;} public String getName(){return name;} }
```

FICHIER COMPLET

tenant-service/src/main/java/com/acme/tenant/domain/TenantRepository.java

```
package com.acme.tenant.domain; import java.util.*; import org.springframework.data.jpa.repository.JpaRepository;  
public interface TenantRepository extends JpaRepository<Tenant,UUID>{ Optional<Tenant> findBySlug(String slug); }
```

FICHER COMPLET

tenant-service/src/main/java/com/acme/tenant/application/TenantUseCase.java

```
package com.acme.tenant.application; import com.acme.tenant.domain.*; import java.util.*; import org.springframework.stereotype.Service; @Service public class TenantUseCase { private final TenantRepository repo; public TenantUseCase(TenantRepository r){repo=r;} public List<Tenant> list(){return repo.findAll();} public Tenant create(String slug,String name){ return repo.save(new Tenant(UUID.randomUUID(),slug,name));} }
```

FICHER COMPLET

tenant-service/src/main/java/com/acme/tenant/web/TenantController.java

```
package com.acme.tenant.web; import com.acme.tenant.application.*; import com.acme.tenant.domain.*; import jakarta.validation.constraints.*; import java.util.*; import org.springframework.http.*; import org.springframework.web.bind.annotation.*; @RestController @RequestMapping("/api/tenants") public class TenantController { record Create(@NotBlank String slug,@NotBlank String name){} private final TenantUseCase use; TenantController(TenantUseCase u){use=u;} @GetMapping List<Tenant> list(){return use.list();} @PostMapping @ResponseStatus(HttpStatus.CREATED) Tenant create(@RequestBody Create c){return use.create(c.slug(),c.name());} }
```

FICHIER COMPLET

tenant-service/src/main/resources/db/migration/V1__tenant.sql

```
CREATE TABLE tenants(id uuid PRIMARY KEY,slug varchar(80) UNIQUE NOT NULL,name varchar(160) NOT NULL);
INSERT INTO tenants(id,slug,name) VALUES ('11111111-1111-1111-1111-111111111111','demo','Demo Industries');
```

FICHER COMPLET

catalog-service/src/main/java/com/acme/catalog/domain/Product.java

```
package com.acme.catalog.domain; import jakarta.persistence.*; import java.math.BigDecimal; import java.util.UUID;
@Entity @Table(name="products",uniqueConstraints=@UniqueConstraint(columnNames={"tenant_id","sku"})) public class
Product { @Id private UUID id; @Column(name="tenant_id",nullable=false) private String tenantId; @Column(
nullable=false) private String sku; @Column(nullable=false) private String name; @Column(nullable=false,
precision=19,scale=2) private BigDecimal price; protected Product(){} public Product(UUID id,String t,String sku,
String n,BigDecimal p){this.id=id;tenantId=t;this.sku=sku;name=n;price=p;} public UUID getId(){return id;} public
String getTenantId(){return tenantId;} public String getSku(){return sku;} public String getName(){return name;}
public BigDecimal getPrice(){return price;} }
```

FICHIER COMPLET

catalog-service/src/main/java/com/acme/catalog/domain/ProductRepository.java

```
package com.acme.catalog.domain; import java.util.*; import org.springframework.data.jpa.repository.JpaRepository;
public interface ProductRepository extends JpaRepository<Product,UUID>{List<Product> findAllByTenantIdOrderByName(
String tenantId); Optional<Product> findByIdAndTenantId(UUID id,String tenantId);}
```

FICHER COMPLET

catalog-service/src/main/java/com/acme/catalog/application/ProductUseCase.java

```
package com.acme.catalog.application; import com.acme.catalog.domain.*; import com.acme.catalog.security.TenantContext; import java.math.BigDecimal; import java.util.*; import org.springframework.stereotype.Service; import org.springframework.transaction.annotation.Transactional; @Service public class ProductUseCase { private final ProductRepository repo; public ProductUseCase(ProductRepository r){repo=r;} @Transactional public Product create(String sku,String name,BigDecimal price){return repo.save(new Product(UUID.randomUUID(),TenantContext.require(),sku,name,price));} @Transactional(readOnly=true) public List<Product> list(){return repo.findAllByTenantIdOrderByName(TenantContext.require());} }
```

FICHIER COMPLET

catalog-service/src/main/java/com/acme/catalog/web/ProductController.java

```
package com.acme.catalog.web; import com.acme.catalog.application.*; import com.acme.catalog.domain.*; import
jakarta.validation.Valid; import jakarta.validation.constraints.*; import java.math.*; import java.util.*; import
org.springframework.http.*; import org.springframework.web.bind.annotation.*;
@RestController @RequestMapping("/api/products") public class ProductController { record Create(@NotBlank String
sku,@NotBlank String name,@DecimalMin("0.01") BigDecimal price){} private final ProductUseCase use;
ProductController(ProductUseCase u){use=u;} @GetMapping List<Product> list(){return use.list();} @PostMapping
@ResponseStatus(HttpStatus.CREATED) Product create(@Valid @RequestBody Create c){return use.create(c.sku(),c.name(
),c.price());} }
```

FICHIER COMPLET

catalog-service/src/main/resources/db/migration/V1__catalog.sql

```
CREATE TABLE products(id uuid PRIMARY KEY,tenant_id varchar(64) NOT NULL,sku varchar(80) NOT NULL,name varchar(200) NOT NULL,price numeric(19,2) NOT NULL CHECK(price>0),UNIQUE(tenant_id,sku));
CREATE INDEX ix_products_tenant ON products(tenant_id);
```

FICHIER COMPLET

inventory-service/src/main/java/com/acme/inventory/domain/Stock.java

```
package com.acme.inventory.domain; import jakarta.persistence.*; import java.util.UUID;
@Entity @Table(name="stocks",uniqueConstraints=@UniqueConstraint(columnNames={"tenant_id","product_id"})) public
class Stock { @Id private UUID id; @Column(name="tenant_id",nullable=false) private String tenantId; @Column(
name="product_id",nullable=false) private UUID productId; @Column(nullable=false) private long available; @Version
private long version; protected Stock(){} public Stock(UUID id,String t,UUID p,long q){this.id=id;tenantId=t;
productId=p;available=q;} public UUID getProductId(){return productId;} public long getAvailable(){return
available;} public void adjust(long delta){if(available+delta<0)throw new IllegalStateException("stock
insuffisant");available+=delta;} }
```

FICHER COMPLET

inventory-service/src/main/java/com/acme/inventory/domain/StockRepository.java

```
package com.acme.inventory.domain; import java.util.*; import org.springframework.data.jpa.repository.*; public  
interface StockRepository extends JpaRepository<Stock,UUID>{ Optional<Stock> findByTenantIdAndProductId(String  
tenant,UUID product); List<Stock> findAllByTenantId(String tenant); }
```

FICHIER COMPLET

inventory-service/src/main/java/com/acme/inventory/application/StockUseCase.java

```
package com.acme.inventory.application; import com.acme.inventory.domain.*; import com.acme.inventory.security.TenantContext; import java.util.*; import org.springframework.stereotype.Service; import org.springframework.transaction.annotation.Transactional;

@Service public class StockUseCase { private final StockRepository repo; public StockUseCase(StockRepository r){ repo=r;} @Transactional public Stock set(UUID product,long qty){String t=TenantContext.require(); Stock s=repo.findByTenantIdAndProductId(t,product).orElseGet()->new Stock(UUID.randomUUID(),t,product,0)); s.adjust(qty-s.getAvailable()); return repo.save(s);} public List<Stock> list(){return repo.findAllByTenantId(TenantContext.require());} }
```

FICHER COMPLET

inventory-service/src/main/java/com/acme/inventory/web/StockController.java

```
package com.acme.inventory.web; import com.acme.inventory.application.*; import com.acme.inventory.domain.*;
import jakarta.validation.constraints.*; import java.util.*; import org.springframework.web.bind.annotation.*;
@RestController @RequestMapping("/api/stocks") public class StockController { record SetStock(@Min(0) long
quantity){} private final StockUseCase use; StockController(StockUseCase u){use=u;} @GetMapping List<Stock> list(){
return use.list();} @PutMapping("/{productId}") Stock set(@PathVariable UUID productId,@RequestBody SetStock r){
return use.set(productId,r.quantity());} }
```

FICHER COMPLET

inventory-service/src/main/resources/db/migration/V1__inventory.sql

```
CREATE TABLE stocks(id uuid PRIMARY KEY,tenant_id varchar(64) NOT NULL,product_id uuid NOT NULL,available bigint NOT NULL CHECK(available>=0),version bigint NOT NULL DEFAULT 0,UNIQUE(tenant_id,product_id)); CREATE INDEX ix_stock_tenant ON stocks(tenant_id);
```

order-service/src/main/java/com/acme/order/domain/PurchaseOrder.java

```
package com.acme.order.domain; import jakarta.persistence.*; import java.math.*; import java.time.*; import java.util.*;

@Entity @Table(name="purchase_orders") public class PurchaseOrder { @Id private UUID id; @Column(name="tenant_id", nullable=false) private String tenantId; @Column(name="product_id", nullable=false) private UUID productId; @Column(nullable=false) private long quantity; @Column(nullable=false,precision=19,scale=2) private BigDecimal total; @Column(nullable=false) private String status; @Column(name="created_at", nullable=false) private Instant createdAt;

protected PurchaseOrder(){} public PurchaseOrder(UUID id,String t,UUID p,long q,BigDecimal total){this.id=id; tenantId=t;productId=p;quantity=q;this.total=total;status="CREATED";createdAt=Instant.now();} public UUID getId(){ return id;} public String getTenantId(){return tenantId;} public UUID getProductId(){return productId;} public long getQuantity(){return quantity;} public BigDecimal getTotal(){return total;} public String getStatus(){return status;} public Instant getCreatedAt(){return createdAt;} }
```

FICHER COMPLET

order-service/src/main/java/com/acme/order/domain/OrderRepository.java

```
package com.acme.order.domain; import java.util.*; import org.springframework.data.jpa.repository.JpaRepository;
public interface OrderRepository extends JpaRepository<PurchaseOrder,UUID>{List<PurchaseOrder>
findAllByTenantIdOrderByCreatedAtDesc(String tenant);}
```

order-service/src/main/java/com/acme/order/infrastructure/OutboxEvent.java

```
package com.acme.order.infrastructure; import jakarta.persistence.*; import java.time.*; import java.util.*;
@Entity @Table(name="outbox_events") public class OutboxEvent { @Id private UUID id; @Column(name="aggregate_id",
nullable=false) private UUID aggregateId; @Column(nullable=false) private String topic; @Column(
columnDefinition="text",nullable=false) private String payload; @Column(name="created_at",nullable=false) private
Instant createdAt; @Column(name="published_at") private Instant publishedAt; protected OutboxEvent(){ public
OutboxEvent(UUID aggregateId,String topic,String payload){id=UUID.randomUUID();this.aggregateId=aggregateId;this.
topic=topic;this.payload=payload;createdAt=Instant.now();} public UUID getId(){return id;} public String getTopic(
){return topic;} public String getPayload(){return payload;} public void published(){publishedAt=Instant.now();} }
```

FICHER COMPLET

order-service/src/main/java/com/acme/order/infrastructure/OutboxRepository.java

```
package com.acme.order.infrastructure; import java.util.*; import org.springframework.data.jpa.repository.*;
import org.springframework.data.repository.query.Param; public interface OutboxRepository extends
JpaRepository<OutboxEvent,UUID>{ @Query(value="select * from outbox_events where published_at is null order by
created_at limit :n for update skip locked",nativeQuery=true) List<OutboxEvent> lockBatch(@Param("n") int n); }
```

order-service/src/main/java/com/acme/order/application/OrderUseCase.java

```
package com.acme.order.application; import com.acme.order.domain.*; import com.acme.order.infrastructure.*; import
com.acme.order.security.TenantContext; import java.math.*; import java.util.*; import org.springframework.
stereotype.Service; import org.springframework.transaction.annotation.Transactional;
@Service public class OrderUseCase { private final OrderRepository orders; private final OutboxRepository outbox;
public OrderUseCase(OrderRepository o, OutboxRepository x){orders=o;outbox=x;} @Transactional public PurchaseOrder
create(UUID product, long qty, BigDecimal unitPrice){String t=TenantContext.require(); PurchaseOrder o=orders.save(
new PurchaseOrder(UUID.randomUUID(),t,product,qty,unitPrice.multiply(BigDecimal.valueOf(qty)))); String json="{\
"eventId\":"\""+UUID.randomUUID()+"\", \"tenantId\":"\""+t+"\", \"orderId\":"\""+o.getId()+"\", \"productId\":"\
\""+product+"\", \"quantity\":"\""+qty+"\""; outbox.save(new OutboxEvent(o.getId(),"orders.created.v1",json)); return o;
} public List<PurchaseOrder> list(){return orders.findAllByTenantIdOrderByCreatedAtDesc(TenantContext.require());}
}
```

FICHER COMPLET

order-service/src/main/java/com/acme/order/infrastructure/OutboxPublisher.java

```
package com.acme.order.infrastructure; import org.springframework.kafka.core.KafkaTemplate; import org.
springframework.scheduling.annotation.*; import org.springframework.stereotype.Component; import org.
springframework.transaction.annotation.Transactional;
@Component @EnableScheduling public class OutboxPublisher { private final OutboxRepository repo; private final
KafkaTemplate<String,String> kafka; public OutboxPublisher(OutboxRepository r,KafkaTemplate<String,String> k){
repo=r;kafka=k;} @Scheduled(fixedDelayString="${outbox.delay-ms:1000}") @Transactional public void publish(){for(
OutboxEvent e:repo.lockBatch(50)){kafka.send(e.getTopic(),e.getId().toString(),e.getPayload()).join();e.published(
);}} }
```

FICHER COMPLET

order-service/src/main/java/com/acme/order/web/OrderController.java

```
package com.acme.order.web; import com.acme.order.application.*; import com.acme.order.domain.*; import jakarta.validation.Valid; import jakarta.validation.constraints.*; import java.math.*; import java.util.*; import org.springframework.http.*; import org.springframework.web.bind.annotation.*; @RestController @RequestMapping("/api/orders") public class OrderController { record Create(@NotNull UUID productId,@Min(1) long quantity,@DecimalMin("0.01") BigDecimal unitPrice){} private final OrderUseCase use; OrderController(OrderUseCase u){use=u;} @PostMapping @ResponseStatus(HttpStatus.CREATED) PurchaseOrder create(@Valid @RequestBody Create c){return use.create(c.productId(),c.quantity(),c.unitPrice());} @GetMapping List<PurchaseOrder> list(){return use.list();} }
```

FICHIER COMPLET

inventory-service/src/main/java/com/acme/inventory/infrastructure/OrderCreatedConsumer.java

```
package com.acme.inventory.infrastructure; import com.acme.inventory.domain.*; import com.fasterxml.jackson.databind.*; import java.util.*; import org.springframework.kafka.annotation.KafkaListener; import org.springframework.stereotype.Component; import org.springframework.transaction.annotation.Transactional; @Component public class OrderCreatedConsumer { private final StockRepository stocks; private final ObjectMapper json; public OrderCreatedConsumer(StockRepository s,ObjectMapper j){stocks=s;j=json;} @KafkaListener(topics="orders.created.v1",groupId="inventory") @Transactional public void consume(String value)throws Exception{ JsonNode n=json.readTree(value); String t=n.get("tenantId").asText(); UUID p=UUID.fromString(n.get("productId").asText()); long q=n.get("quantity").asLong(); Stock s=stocks.findByTenantIdAndProductId(t,p).orElseThrow(); s.adjust(-q);} }
```

FICHIER COMPLET

order-service/src/main/resources/db/migration/V1__orders.sql

```
CREATE TABLE purchase_orders(id uuid PRIMARY KEY,tenant_id varchar(64) NOT NULL,product_id uuid NOT NULL,quantity  
bigint NOT NULL CHECK(quantity>0),total numeric(19,2) NOT NULL,status varchar(30) NOT NULL,created_at timestamptz  
NOT NULL); CREATE INDEX ix_order_tenant ON purchase_orders(tenant_id,created_at DESC);  
CREATE TABLE outbox_events(id uuid PRIMARY KEY,aggregate_id uuid NOT NULL,topic varchar(120) NOT NULL,payload text  
NOT NULL,created_at timestamptz NOT NULL,published_at timestamptz); CREATE INDEX ix_outbox_pending ON  
outbox_events(created_at) WHERE published_at IS NULL;
```

FICHER COMPLET

catalog-service/src/test/java/com/acme/catalog/ProductUseCaseTest.java

```
package com.acme.catalog; import com.acme.catalog.application.*; import com.acme.catalog.domain.*; import com.acme.catalog.security.TenantContext; import java.math.*; import org.junit.jupiter.api.*; import static org.mockito.Mockito.*; import static org.assertj.core.api.Assertions.*;
class ProductUseCaseTest { ProductRepository repo=mock(ProductRepository.class); ProductUseCase use=new ProductUseCase(repo); @AfterEach void clear(){TenantContext.clear();} @Test void tenantEstImpose(){TenantContext.set("demo"); use.create("SKU-1", "Clavier", new BigDecimal("49.90")); verify(repo).save(argThat(p->p.getTenantId().equals("demo")));} @Test void refuseSansTenant(){assertThatThrownBy(()->use.list()).isInstanceOf(IllegalStateException.class);} }
```

FICHER COMPLET

order-service/src/test/java/com/acme/order/OrderUseCaseTest.java

```
package com.acme.order; import com.acme.order.application.*; import com.acme.order.domain.*; import com.acme.order.
infrastructure.*; import com.acme.order.security.TenantContext; import java.math.*; import java.util.*; import org.
junit.jupiter.api.*; import static org.mockito.Mockito.*;
class OrderUseCaseTest { @Test void commandeEtOutboxSontEcrits(){OrderRepository o=mock(OrderRepository.class);
OutboxRepository x=mock(OutboxRepository.class); when(o.save(any())).thenReturn(i->i.getArgument(0));
TenantContext.set("demo"); try{new OrderUseCase(o,x).create(UUID.randomUUID(),2,new BigDecimal("10.00")); verify(
o).save(any());verify(x).save(any());}finally{TenantContext.clear();}} }
```

FICHIER COMPLET

catalog-service/src/test/java/com/acme/catalog/ProductApiIT.java

```
package com.acme.catalog; import org.junit.jupiter.api.*; import org.springframework.boot.test.context.*  
SpringBootTest; import org.testcontainers.containers.PostgreSQLContainer; import org.testcontainers.junit.jupiter.*  
*;  
@Testcontainers @SpringBootTest class ProductApiIT { @Container static PostgreSQLContainer<?> db=new  
PostgreSQLContainer<>("postgres:18-alpine"); @Test void contexteDemarre(){Assertions.assertTrue(db.isRunning());} }
```

FICHIER COMPLET

web/package.json

```
{
  "name": "b2b-web",
  "version": "0.1.0",
  "private": true,
  "scripts": {
    "start": "ng serve --proxy-config proxy.conf.json",
    "build": "ng build",
    "test": "ng test --watch=false"
  },
  "dependencies": {
    "@angular/animations": "22.0.0",
    "@angular/common": "22.0.0",
    "@angular/compiler": "22.0.0",
    "@angular/core": "22.0.0",
    "@angular/forms": "22.0.0",
    "@angular/platform-browser": "22.0.0",
    "@angular/router": "22.0.0",
    "angular-auth-oidc-client": "^20.0.0",
    "rxjs": "^7.8.2",
    "zone.js": "^0.15.1"
  },
  "devDependencies": {
    "@angular-devkit/build-angular": "22.0.0",
    "@angular/cli": "22.0.0",
    "@angular/compiler-cli": "22.0.0",
    "typescript": "^5.9.0",
    "jasmine-core": "5.10.0",
    "karma": "^6.4.4",
    "karma-chrome-launcher": "^3.2.0"
  }
}
```

FICHER COMPLET

web/angular.json

```
{ "$schema": "./node_modules/@angular/cli/lib/config/schema.json", "version": 1, "projects": { "b2b-web": { "projectType": "application", "root": "", "sourceRoot": "src", "architect": { "build": { "builder": "@angular-devkit/build-angular:application", "options": { "browser": "src/main.ts", "index": "src/index.html", "tsConfig": "tsconfig.app.json", "styles": [ "src/styles.css" ], "assets": [ ] }, "serve": { "builder": "@angular-devkit/build-angular:dev-server", "configurations": { "production": { "buildTarget": "b2b-web:build:production" } }, "test": { "builder": "@angular-devkit/build-angular:karma", "options": { "tsConfig": "tsconfig.spec.json" } } } } }
```

FICHER COMPLET

web/tsconfig.json

```
{
  "compileOnSave": false,
  "compilerOptions": {
    "baseUrl": "./",
    "outDir": "./dist/out-tsc",
    "strict": true,
    "noImplicitOverride": true,
    "noPropertyAccessFromIndexSignature": true,
    "noImplicitReturns": true,
    "noFallthroughCasesInSwitch": true,
    "sourceMap": true,
    "declaration": false,
    "experimentalDecorators": true,
    "moduleResolution": "bundler",
    "target": "ES2022",
    "module": "ES2022",
    "lib": ["ES2022", "dom"],
    "angularCompilerOptions": {
      "strictInjectionParameters": true,
      "strictInputAccessModifiers": true,
      "strictTemplates": true
    }
  }
}
```

FICHIER COMPLET

web/tsconfig.app.json

```
{ "extends": "../tsconfig.json", "compilerOptions": { "outDir": "../out-tsc/app", "types": [], "files": [ "src/main.ts",  
  "include": [ "src/**/*.d.ts" ] }
```

FICHIER COMPLET

web/tsconfig.spec.json

```
{ "extends": "../tsconfig.json", "compilerOptions": { "outDir": "../out-tsc/spec", "types": [ "jasmine" ] }, "include": [ "src/**/*.spec.ts", "src/**/*.d.ts" ] }
```

FICHIER COMPLET

web/src/index.html

```
<!doctype html><html lang="fr"><head><meta charset="utf-8"><title>B2B Commerce</title><base href="/"><meta  
name="viewport" content="width=device-width,initial-scale=1"></head><body><app-root></app-root></body></html>
```

FICHER COMPLET

web/src/styles.css

```
body{font-family:system-ui;margin:0;background:#f4f6f8;color:#172b4d}main{max-width:960px;margin:auto;padding:24px}nav{display:flex;gap:16px;background:#12324a;padding:14px}nav a{color:white}form{display:grid;gap:8px;max-width:420px}input,button{padding:10px}article{background:white;padding:12px;margin:8px 0;border-radius:6px}.error{color:#b42318}
```

FICHER COMPLET

web/src/main.ts

```
import {bootstrapApplication} from '@angular/platform-browser'; import {provideHttpClient,withInterceptors} from
'@angular/common/http'; import {provideRouter} from '@angular/router'; import {AppComponent} from './app/app.
component'; import {routes} from './app/app.routes'; import {authInterceptor} from './app/auth.interceptor';
bootstrapApplication(AppComponent,{providers:[provideRouter(routes),provideHttpClient(withInterceptors([
authInterceptor]))])).catch(console.error);
```

web/src/app/auth.service.ts

Pour un vrai flux Authorization Code + PKCE, utiliser la bibliotheque OIDC choisie pour generer verifier/state/code_verifier. Le walking skeleton montre les points d'integration sans secret client.

```
import {Injectable, signal} from '@angular/core';
@Injectable({providedIn: 'root'}) export class AuthService { readonly token=signal(sessionStorage.getItem(
'access_token')); login(){const callback=encodeURIComponent(location.origin+'/callback');location.href='http://
localhost:8081/realms/b2b/protocol/openid-connect/auth?client_id=b2b-
web&response_type=code&scope=openid&redirect_uri='+callback+'&code_challenge_method=S256&code_challenge=CLI_GENERAT
ED_PKCE_CHALLENGE';} logout(){sessionStorage.removeItem('access_token');this.token.set(null);} }
```

FICHER COMPLET

web/src/app/auth.interceptor.ts

```
import {HttpInterceptorFn} from '@angular/common/http'; import {inject} from '@angular/core'; import {AuthService}
from './auth.service';
export const authInterceptor:HttpInterceptorFn=(req,next)=>{const token=inject(AuthService).token();return next(
token?req.clone({setHeaders:{Authorization:`Bearer ${token}`}}):req);};
```

FICHER COMPLET

web/src/app/api.service.ts

```
import {HttpClient} from '@angular/common/http'; import {Injectable} from '@angular/core'; import {Observable}
from 'rxjs';
export interface Product{id:string;sku:string;name:string;price:number} export interface Stock{productId:string;
available:number} export interface Order{id:string;productId:string;quantity:number;total:number;status:string}
@Injectable({providedIn:'root'}) export class ApiService{constructor(private http:HttpClient){} products():
Observable<Product[]>{return this.http.get<Product[]>('/api/products')} createProduct(v:{sku:string;name:string;
price:number}){return this.http.post<Product>('/api/products',v)} stocks(){return this.http.get<Stock[]>('/api/
stocks')} setStock(id:string,quantity:number){return this.http.put<Stock>(`/api/stocks/${id}`, {quantity})} orders(
){return this.http.get<Order[]>('/api/orders')} createOrder(v:{productId:string;quantity:number;unitPrice:number}){
return this.http.post<Order>('/api/orders',v)}
```

FICHER COMPLET

web/src/app/app.routes.ts

```
import {Routes} from '@angular/router'; import {ProductsComponent} from './products.component'; import {
OrdersComponent} from './orders.component';
export const routes:Routes=[{path:'',pathMatch:'full',redirectTo:'products'},{path:'products',component:
ProductsComponent},{path:'orders',component:OrdersComponent}];
```

FICHER COMPLET

web/src/app/app.component.ts

```
import {Component} from '@angular/core'; import {RouterLink,RouterOutlet} from '@angular/router'; import {
AuthService} from '../auth.service';
@Component({selector:'app-root',standalone:true,imports:[RouterLink,RouterOutlet],template:`<nav><a routerLink="/
products">Produits</a><a routerLink="/orders">Commandes</a><button (click)="auth.login()">Connexion Keycloak</
button></nav><main><h1>Demo Industries</h1><router-outlet/></main>`) export class AppComponent{constructor(public
auth:AuthService){}}
```

web/src/app/products.component.ts

```
import {Component,OnInit} from '@angular/core'; import {CommonModule} from '@angular/common'; import {FormsModule}
from '@angular/forms'; import {ApiService,Product} from '../api.service';
@Component({standalone:true,imports:[CommonModule,FormsModule],template:`<h2>Catalogue</h2><form (
ngSubmit)="create()"><input name="sku" [(ngModel)]="draft.sku" placeholder="SKU" required><input name="name" [(
ngModel)]="draft.name" placeholder="Nom" required><input name="price" type="number" [(ngModel)]="draft.price"
min=".01" required><button>Creer</button></form><article *ngFor="let p of products">{{p.sku}} - {{p.name}} - {{p.
price|currency:'EUR'}}</article>`}) export class ProductsComponent implements OnInit{products:Product[]=[];draft={
sku:'',name:'',price:1};constructor(private api:ApiService){ngOnInit(){this.reload()}reload(){this.api.products().
subscribe(v=>this.products=v)}create(){this.api.createProduct(this.draft).subscribe(=>{this.draft={sku:'',name:
'',price:1};this.reload()})}}
```

FICHIER COMPLET

web/src/app/orders.component.ts

```
import {Component,OnInit} from '@angular/core'; import {CommonModule} from '@angular/common'; import {FormsModule}
from '@angular/forms'; import {ApiService,Order,Product} from './api.service';
@Component({standalone:true,imports:[CommonModule,FormsModule],template:`<h2>Commandes</h2><form (
ngSubmit)="create()"><select name="product" [(ngModel)]="draft.productId"><option *ngFor="let p of products" [
value]="p.id">{{p.name}}</option></select><input name="quantity" type="number" [(ngModel)]="draft.quantity" [
min="1"><input name="price" type="number" [(ngModel)]="draft.unitPrice" min=".01"><button>Commander</button></
form><article *ngFor="let o of orders">{{o.id}} - {{o.quantity}} - {{o.total|currency:'EUR'}} - {{o.status}}</
article>`}) export class OrdersComponent implements OnInit{orders:Order[]=[];products:Product[]=[];draft={
productId:'',quantity:1,unitPrice:1};constructor(private api:ApiService){}ngOnInit(){this.api.products().subscribe(
p=>{this.products=p;if(p[0])this.draft.productId=p[0].id});this.reload()}reload(){this.api.orders().subscribe(
v=>this.orders=v)}create(){this.api.createOrder(this.draft).subscribe(()=>this.reload())}}
```

FICHER COMPLET

web/src/app/products.component.spec.ts

```
import { TestBed } from '@angular/core/testing'; import { provideHttpClient } from '@angular/common/http'; import {
ProductsComponent } from './products.component'; describe('ProductsComponent', () => { it('se cree', () => { TestBed.
configureTestingModule({ imports: [ProductsComponent], providers: [provideHttpClient()] }); expect(TestBed.
createComponent(ProductsComponent).componentInstance).toBeTruthy(); }); });
```

FICHIER COMPLET

mobile/pubspec.yaml

```
name: b2b_mobile
description: Client mobile du MVP B2B
publish_to: none
version: 0.1.0+1
environment:
  sdk: '>=3.8.0 <4.0.0'
dependencies:
  flutter:
    sdk: flutter
  http: ^1.5.0
  flutter_appauth: ^11.0.0
  flutter_secure_storage: ^9.2.4
dev_dependencies:
  flutter_test:
    sdk: flutter
  flutter_lints: ^6.0.0
flutter:
  uses-material-design: true
```

FICHER COMPLET

mobile/lib/config.dart

```
class Config { static const api=String.fromEnvironment('API_URL',defaultValue:'http://10.0.2.2:8080'); static  
const issuer=String.fromEnvironment('ISSUER',defaultValue:'http://10.0.2.2:8081/realms/b2b'); static const  
clientId='b2b-mobile'; static const redirect='com.acme.b2b/oauthredirect'; }
```

FICHER COMPLET

mobile/lib/models.dart

```
class Product{final String id,sku,name;final double price;Product.fromJson(Map<String,dynamic> j):id=j['id'],sku=j['sku'],name=j['name'],price=(j['price'] as num).toDouble();} class Order{final String id,status;final int quantity;final double total;Order.fromJson(Map<String,dynamic> j):id=j['id'],status=j['status'],quantity=j['quantity'],total=(j['total'] as num).toDouble();}
```

FICHER COMPLET

mobile/lib/auth_service.dart

```
import 'package:flutter_appauth/flutter_appauth.dart'; import 'package:flutter_secure_storage/
flutter_secure_storage.dart'; import 'config.dart';
class AuthService{final _oidc=const FlutterAppAuth();final _store=const FlutterSecureStorage();Future<void> login(
)async{final r=await _oidc.authorizeAndExchangeCode(AuthorizationTokenRequest(Config.clientId,Config.redirect,
issuer:Config.issuer,scopes:['openid','profile']));if(r?.accessToken!=null)await _store.write(key:'token',value:r!.
accessToken);}Future<String?> token()=>_store.read(key:'token');Future<void> logout()=>_store.deleteAll();}
```

FICHIER COMPLET

mobile/lib/api_client.dart

```
import 'dart:convert'; import 'package:http/http.dart' as http; import 'auth_service.dart'; import 'config.dart';
import 'models.dart';
class ApiClient{final AuthService auth;ApiClient(this.auth);Future<Map<String,String>> _headers()async=>{'Content-
Type':'application/json','Authorization':'Bearer ${await auth.token()}}';Future<List<Product>> products()async{
final r=await http.get(Uri.parse('${Config.api}/api/products'),headers:await _headers());_ok(r);return (jsonDecode(
r.body) as List).map((e)=>Product.fromJson(e)).toList();}Future<List<Order>> orders()async{final r=await http.get(
Uri.parse('${Config.api}/api/orders'),headers:await _headers());_ok(r);return (jsonDecode(r.body) as List).map((
e)=>Order.fromJson(e)).toList();}Future<void> order(Product p,int q)async{final r=await http.post(Uri.parse('${
Config.api}/api/orders'),headers:await _headers(),body:jsonEncode({'productId':p.id,'quantity':q,'unitPrice':p.
price}));_ok(r);}void _ok(http.Response r){if(r.statusCode<200||r.statusCode>=300)throw Exception('HTTP ${r.
statusCode}: ${r.body}');}}
```

FICHIER COMPLET

mobile/lib/main.dart

```
import 'package:flutter/material.dart'; import 'api_client.dart'; import 'auth_service.dart'; import 'product_page.dart';
void main()=>runApp(const B2bApp());class B2bApp extends StatelessWidget{const B2bApp({super.key});@override
Widget build(BuildContext context){final auth=AuthService();return MaterialApp(title:'B2B Commerce',theme:
ThemeData(colorSchemeSeed:Colors.blue,useMaterial3:true),home:FutureBuilder<String?>(future:auth.token(),builder:(
c,s)=>s.data==null?Scaffold(body:Center(child:FilledButton(onPressed:())async{await auth.login();if(c.mounted)(c as
Element).markNeedsBuild();},child:const Text('Connexion Keycloak'))):ProductPage(api:ApiClient(auth)));}}
```

FICHIER COMPLET

mobile/lib/product_page.dart

```
import 'package:flutter/material.dart'; import 'api_client.dart'; import 'models.dart'; import 'order_page.dart';
class ProductPage extends StatelessWidget{final ApiClient api;const ProductPage({super.key,required this.api});
@override Widget build(BuildContext c)=>Scaffold(appBar:AppBar(title:const Text('Produits'),actions:[IconButton(
icon:const Icon(Icons.receipt),onPressed:()=>Navigator.push(c,MaterialPageRoute(builder:(_)=>OrderPage(api:
api)))]),body:FutureBuilder<List<Product>>(future:api.products(),builder:(c,s){if(s.hasError)return Text('${s.
error}');if(!s.hasData)return const Center(child:CircularProgressIndicator());return ListView(children:s.data!.map(
(p)=>ListTile(title:Text(p.name),subtitle:Text(p.sku),trailing:FilledButton(onPressed:()=>api.order(p,1),child:
Text('${p.price.toStringAsFixed(2)} EUR')))).toList();}));}
```

FICHER COMPLET

mobile/lib/order_page.dart

```
import 'package:flutter/material.dart'; import 'api_client.dart'; import 'models.dart';class OrderPage extends StatelessWidget{final ApiClient api;const OrderPage({super.key,required this.api});@override Widget build(BuildContext c)=>Scaffold(appBar:AppBar(title:const Text('Commandes')),body:FutureBuilder<List<Order>>(future:api.orders(),builder:(c,s){if(s.hasError)return Text('${s.error}');if(!s.hasData)return const Center(child: CircularProgressIndicator());return ListView(children:s.data!.map((o)=>ListTile(title:Text(o.id),subtitle:Text('${o.quantity} - ${o.status}'),trailing:Text('${o.total.toStringAsFixed(2)} EUR'))).toList());}));}
```

FICHER COMPLET

mobile/test/widget_test.dart

```
import 'package:flutter_test/flutter_test.dart'; import 'package:b2b_mobile/models.dart'; void main(){test('parse  
produit',(){final p=Product.fromJson({'id':'1','sku':'A','name':'Article','price':12.5});expect(p.price,12.5);  
expect(p.sku,'A');});});}
```

FICHIER COMPLET

docker-compose.yml

```
name: b2b-mvp
services:
  postgres:
    image: postgres:18-alpine
    environment: {POSTGRES_USER: b2b, POSTGRES_PASSWORD: local-only-change-me}
    volumes: [./infra/postgres/init.sql:/docker-entrypoint-initdb.d/init.sql:ro, pgdata:/var/lib/postgresql/data]
    healthcheck: {test: [CMD-SHELL, pg_isready -U b2b], interval: 5s, timeout: 3s, retries: 20}
  kafka:
    image: apache/kafka:4.3.0
    environment: {KAFKA_NODE_ID: 1, KAFKA_PROCESS_ROLES: broker,controller, KAFKA_LISTENERS: 'PLAINTEXT://:9092,
CONTROLLER://:9093', KAFKA_ADVERTISED_LISTENERS: 'PLAINTEXT://kafka:9092', KAFKA_CONTROLLER_LISTENER_NAMES:
CONTROLLER, KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: 'CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT',
KAFKA_CONTROLLER_QUORUM_VOTERS: '1@kafka:9093', KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1}
  keycloak:
    image: quay.io/keycloak/keycloak:26.7
    command: start-dev --import-realm
    environment: {KC_BOOTSTRAP_ADMIN_USERNAME: admin, KC_BOOTSTRAP_ADMIN_PASSWORD: local-only-change-me}
    volumes: [./infra/keycloak/realm-b2b.json:/opt/keycloak/data/import/realm.json:ro]
    ports: ['8081:8080']
  gateway: {build: ./gateway, ports: ['8080:8080'], depends_on: [keycloak]}
  tenant-service: {build: ./tenant-service, depends_on: [postgres,kafka,keycloak]}
  catalog-service: {build: ./catalog-service, depends_on: [postgres,kafka,keycloak]}
  inventory-service: {build: ./inventory-service, depends_on: [postgres,kafka,keycloak]}
  order-service: {build: ./order-service, depends_on: [postgres,kafka,keycloak]}
  web: {build: ./web, ports: ['4200:80'], depends_on: [gateway]}
volumes: {pgdata: {}}
```

infra/keycloak/realm-b2b.json

Pour un vrai flux Authorization Code + PKCE, utiliser la bibliothèque OIDC choisie pour generer verifier/state/code_verifier. Le walking skeleton montre les points d'integration sans secret client.

```
{
  "realm": "b2b",
  "enabled": true,
  "registrationAllowed": false,
  "roles": {
    "realm": [
      { "name": "buyer" },
      { "name": "catalog-admin" }
    ]
  },
  "clients": [
    {
      "clientId": "b2b-web",
      "publicClient": true,
      "standardFlowEnabled": true,
      "redirectUris": [ "http://localhost:4200/*" ],
      "webOrigins": [ "http://localhost:4200" ],
      "attributes": {
        "pkce.code.challenge.method": "S256"
      },
      "clientId": "b2b-mobile",
      "publicClient": true,
      "standardFlowEnabled": true,
      "redirectUris": [ "com.acme.b2b:/oauthredirect" ],
      "attributes": {
        "pkce.code.challenge.method": "S256"
      },
      "users": [
        {
          "username": "buyer@demo.test",
          "enabled": true,
          "emailVerified": true,
          "attributes": {
            "tenant_id": [ "demo" ]
          },
          "credentials": [
            {
              "type": "password",
              "value": "local-only-change-me",
              "temporary": true
            }
          ],
          "realmRoles": [ "buyer" ]
        }
      ]
    }
  ]
}
```

FICHIER COMPLET

gateway/Dockerfile

```
FROM maven:3.9.11-eclipse-temurin-25 AS build
WORKDIR /src
COPY pom.xml .
COPY src src
RUN mvn -B -DskipTests package
FROM eclipse-temurin:25-jre
RUN useradd -r -u 10001 app
USER app
COPY --from=build /src/target/*.jar /app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-XX:MaxRAMPercentage=75", "-jar", "/app.jar"]
```

FICHIER COMPLET

tenant-service/Dockerfile

```
FROM maven:3.9.11-eclipse-temurin-25 AS build
WORKDIR /src
COPY pom.xml .
COPY src src
RUN mvn -B -DskipTests package
FROM eclipse-temurin:25-jre
RUN useradd -r -u 10001 app
USER app
COPY --from=build /src/target/*.jar /app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-XX:MaxRAMPercentage=75", "-jar", "/app.jar"]
```

FICHIER COMPLET

catalog-service/Dockerfile

```
FROM maven:3.9.11-eclipse-temurin-25 AS build
WORKDIR /src
COPY pom.xml .
COPY src src
RUN mvn -B -DskipTests package
FROM eclipse-temurin:25-jre
RUN useradd -r -u 10001 app
USER app
COPY --from=build /src/target/*.jar /app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-XX:MaxRAMPercentage=75", "-jar", "/app.jar"]
```

FICHIER COMPLET

inventory-service/Dockerfile

```
FROM maven:3.9.11-eclipse-temurin-25 AS build
WORKDIR /src
COPY pom.xml .
COPY src src
RUN mvn -B -DskipTests package
FROM eclipse-temurin:25-jre
RUN useradd -r -u 10001 app
USER app
COPY --from=build /src/target/*.jar /app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-XX:MaxRAMPercentage=75", "-jar", "/app.jar"]
```

FICHIER COMPLET

order-service/Dockerfile

```
FROM maven:3.9.11-eclipse-temurin-25 AS build
WORKDIR /src
COPY pom.xml .
COPY src src
RUN mvn -B -DskipTests package
FROM eclipse-temurin:25-jre
RUN useradd -r -u 10001 app
USER app
COPY --from=build /src/target/*.jar /app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-XX:MaxRAMPercentage=75", "-jar", "/app.jar"]
```

FICHIER COMPLET

web/Dockerfile

```
FROM node:24-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build
FROM nginx:1.29-alpine
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=build /app/dist/b2b-web/browser /usr/share/nginx/html
USER 101
```

FICHIER COMPLET

web/nginx.conf

```
server { listen 8080; root /usr/share/nginx/html; location /api/ { proxy_pass http://gateway:8080;  
proxy_set_header Authorization $http_authorization; } location / { try_files $uri $uri/ /index.html; } }
```

FICHIER COMPLET

infra/k8s/namespace.yaml

```
apiVersion: v1
kind: Namespace
metadata: {name: b2b}
```

FICHIER COMPLET

infra/k8s/configmap.yaml

```
apiVersion: v1
kind: ConfigMap
metadata: {name: b2b-config, namespace: b2b}
data: {ISSUER_URI: 'http://keycloak:8080/realms/b2b', KAFKA_BOOTSTRAP: 'kafka:9092', DB_USER: b2b}
```

FICHER COMPLET

infra/k8s/secrets.example.yaml

```
apiVersion: v1
kind: Secret
metadata: {name: b2b-secrets, namespace: b2b}
type: Opaque
stringData: {DB_PASSWORD: REPLACE_VIA_SECRET_MANAGER, KC_ADMIN_PASSWORD: REPLACE_VIA_SECRET_MANAGER}
```

FICHIER COMPLET

infra/k8s/postgres.yaml

```
apiVersion: apps/v1
kind: StatefulSet
metadata: {name: postgres, namespace: b2b}
spec:
  serviceName: postgres
  replicas: 1
  selector: {matchLabels: {app: postgres}}
  template:
    metadata: {labels: {app: postgres}}
    spec:
      containers:
        - name: postgres
          image: postgres:18-alpine
          ports: [{containerPort: 5432}]
          env: [{name: POSTGRES_USER,value: b2b},{name: POSTGRES_PASSWORD,valueFrom:{secretKeyRef:{name:b2b-secrets,
key:DB_PASSWORD}}}]]
          volumeMounts: [{name: data,mountPath: /var/lib/postgresql/data}]
      volumeClaimTemplates: [{metadata:{name:data},spec:{accessModes:[ReadWriteOnce],resources:{requests:{storage:
5Gi}}}}]]
---
apiVersion: v1
kind: Service
metadata: {name: postgres, namespace: b2b}
spec: {selector: {app: postgres}, ports: [{port: 5432}]}
```

FICHIER COMPLET

infra/k8s/kafka.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata: {name: kafka, namespace: b2b}
spec: {replicas: 1, selector: {matchLabels: {app: kafka}}, template: {metadata: {labels: {app: kafka}}, spec: {
containers: [{name: kafka, image: 'apache/kafka:4.3.0', ports: [{containerPort: 9092}], env: [{name: KAFKA_NODE_ID,
value: '1'}, {name: KAFKA_PROCESS_ROLES, value: 'broker,controller'}, {name: KAFKA_LISTENERS, value: 'PLAINTEXT://:
9092,CONTROLLER://:9093'}, {name: KAFKA_CONTROLLER_QUORUM_VOTERS, value: '1@localhost:9093'}]}]}]}
---
apiVersion: v1
kind: Service
metadata: {name: kafka, namespace: b2b}
spec: {selector: {app: kafka}, ports: [{port: 9092}]}
```

FICHIER COMPLET

infra/k8s/keycloak.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata: {name: keycloak, namespace: b2b}
spec: {replicas: 1, selector: {matchLabels: {app: keycloak}}, template: {metadata: {labels: {app: keycloak}}, spec:
  {containers: [{name: keycloak, image: 'quay.io/keycloak/keycloak:26.7', args: [start-dev], ports: [{containerPort:
    8080}], readinessProbe: {httpGet: {path: /realms/master,port: 8080}}}}}}
---
apiVersion: v1
kind: Service
metadata: {name: keycloak, namespace: b2b}
spec: {selector: {app: keycloak}, ports: [{port: 8080}]}
```

FICHER COMPLET

infra/k8s/ingress.yaml

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata: {name: b2b, namespace: b2b}
spec:
  rules:
    - host: b2b.local
      http:
        paths:
          - {path: /api, pathType: Prefix, backend: {service: {name: gateway, port: {number: 8080}}}}
          - {path: /, pathType: Prefix, backend: {service: {name: web, port: {number: 8080}}}}
```

FICHIER COMPLET

infra/k8s/gateway.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata: {name: gateway, namespace: b2b}
spec:
  replicas: 1
  selector: {matchLabels: {app: gateway}}
  template:
    metadata: {labels: {app: gateway}}
    spec:
      securityContext: {runAsNonRoot: true}
      containers:
        - name: gateway
          image: ghcr.io/acme/gateway:0.1.0
          ports: [{containerPort: 8080}]
          envFrom: [{configMapRef: {name: b2b-config}}, {secretRef: {name: b2b-secrets}}]
          readinessProbe: {httpGet: {path: /actuator/health, port: 8080}, initialDelaySeconds: 20}
          resources: {requests: {cpu: 100m, memory: 256Mi}, limits: {memory: 768Mi}}
---
apiVersion: v1
kind: Service
metadata: {name: gateway, namespace: b2b}
spec: {selector: {app: gateway}, ports: [{port: 8080}]}
```

FICHIER COMPLET

infra/k8s/services.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata: {name: tenant-service, namespace: b2b}
spec:
  replicas: 1
  selector: {matchLabels: {app: tenant-service}}
  template:
    metadata: {labels: {app: tenant-service}}
    spec:
      securityContext: {runAsNonRoot: true}
      containers:
        - name: tenant-service
          image: ghcr.io/acme/tenant-service:0.1.0
          ports: [{containerPort: 8080}]
          envFrom: [{configMapRef: {name: b2b-config}}, {secretRef: {name: b2b-secrets}}]
          readinessProbe: {httpGet: {path: /actuator/health, port: 8080}, initialDelaySeconds: 20}
          resources: {requests: {cpu: 100m, memory: 256Mi}, limits: {memory: 768Mi}}
---
apiVersion: v1
kind: Service
metadata: {name: tenant-service, namespace: b2b}
spec: {selector: {app: tenant-service}, ports: [{port: 8080}]}
---
apiVersion: apps/v1
kind: Deployment
metadata: {name: catalog-service, namespace: b2b}
spec:
  replicas: 1
  selector: {matchLabels: {app: catalog-service}}
  template:
    metadata: {labels: {app: catalog-service}}
    spec:
      securityContext: {runAsNonRoot: true}
      containers:
        - name: catalog-service
          image: ghcr.io/acme/catalog-service:0.1.0
          ports: [{containerPort: 8080}]
          envFrom: [{configMapRef: {name: b2b-config}}, {secretRef: {name: b2b-secrets}}]
          readinessProbe: {httpGet: {path: /actuator/health, port: 8080}, initialDelaySeconds: 20}
          resources: {requests: {cpu: 100m, memory: 256Mi}, limits: {memory: 768Mi}}
---
apiVersion: v1
kind: Service
metadata: {name: catalog-service, namespace: b2b}
spec: {selector: {app: catalog-service}, ports: [{port: 8080}]}
---
apiVersion: apps/v1
kind: Deployment
metadata: {name: inventory-service, namespace: b2b}
spec:
  replicas: 1
  selector: {matchLabels: {app: inventory-service}}
  template:
    metadata: {labels: {app: inventory-service}}
    spec:
      securityContext: {runAsNonRoot: true}
      containers:
        - name: inventory-service
          image: ghcr.io/acme/inventory-service:0.1.0
          ports: [{containerPort: 8080}]
          envFrom: [{configMapRef: {name: b2b-config}}, {secretRef: {name: b2b-secrets}}]
          readinessProbe: {httpGet: {path: /actuator/health, port: 8080}, initialDelaySeconds: 20}
          resources: {requests: {cpu: 100m, memory: 256Mi}, limits: {memory: 768Mi}}
---
apiVersion: v1
kind: Service
metadata: {name: inventory-service, namespace: b2b}
spec: {selector: {app: inventory-service}, ports: [{port: 8080}]}
---
apiVersion: apps/v1
kind: Deployment
metadata: {name: order-service, namespace: b2b}
spec:
  replicas: 1
  selector: {matchLabels: {app: order-service}}
  template:
    metadata: {labels: {app: order-service}}
    spec:
      securityContext: {runAsNonRoot: true}
      containers:
        - name: order-service
          image: ghcr.io/acme/order-service:0.1.0
          ports: [{containerPort: 8080}]
          envFrom: [{configMapRef: {name: b2b-config}}, {secretRef: {name: b2b-secrets}}]
          readinessProbe: {httpGet: {path: /actuator/health, port: 8080}, initialDelaySeconds: 20}
          resources: {requests: {cpu: 100m, memory: 256Mi}, limits: {memory: 768Mi}}
---
apiVersion: v1
kind: Service
metadata: {name: order-service, namespace: b2b}
spec: {selector: {app: order-service}, ports: [{port: 8080}]}
```

FICHIER COMPLET

infra/k8s/web.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata: {name: web, namespace: b2b}
spec:
  replicas: 1
  selector: {matchLabels: {app: web}}
  template:
    metadata: {labels: {app: web}}
    spec:
      securityContext: {runAsNonRoot: true}
      containers:
      - name: web
        image: ghcr.io/acme/web:0.1.0
        ports: [{containerPort: 8080}]
        envFrom: [{configMapRef: {name: b2b-config}}, {secretRef: {name: b2b-secrets}}]
        readinessProbe: {httpGet: {path: /actuator/health, port: 8080}, initialDelaySeconds: 20}
        resources: {requests: {cpu: 100m, memory: 256Mi}, limits: {memory: 768Mi}}
---
apiVersion: v1
kind: Service
metadata: {name: web, namespace: b2b}
spec: {selector: {app: web}, ports: [{port: 8080}]}
```

FICHER COMPLET

scripts/dev-up.ps1

```
$ErrorActionPreference='Stop'  
if(!(Test-Path .env)){Copy-Item .env.example .env}  
docker compose config  
docker compose up -d --build  
docker compose ps
```

FICHER COMPLET

scripts/dev-down.ps1

```
$ErrorActionPreference='Stop'  
docker compose down --remove-orphans
```

FICHIER COMPLET

scripts/verify.ps1

```
$ErrorActionPreference='Stop'  
mvn -T 1C clean verify  
npm --prefix web ci  
npm --prefix web run test -- --watch=false  
npm --prefix web run build  
flutter pub get --directory mobile  
flutter test mobile  
docker compose config | Out-Null  
Write-Host 'Verification terminee' -ForegroundColor Green
```

FICHIER COMPLET

scripts/kind-up.ps1

```
$ErrorActionPreference='Stop'  
kind create cluster --name b2b-mvp  
kubectl apply -f infra/k8s/namespace.yaml  
kubectl apply -f infra/k8s/  
kubectl -n b2b get pods  
kubectl -n b2b rollout status deployment/gateway --timeout=180s
```

FICHIER COMPLET

README.md

B2B Commerce MVP

Walking skeleton multi-tenant: Keycloak -> gateway -> catalogue/stock/commandes -> PostgreSQL + Kafka Outbox.

Demarrage

`Copy-Item .env.example .env; docker compose up -d --build`

Demo

Compte `buyer@demo.test`; le mot de passe local est temporaire. Tenant logique: `demo`. Creer un produit, fixer le stock, puis creer une commande. L'order-service ecrit commande et outbox dans la meme transaction; le publisher envoie `orders.created.v1`; inventory consomme et decremente.

Production

Ne jamais committer `.env` ou un Secret reel. Utiliser TLS, hostname Keycloak public stable, migrations controlees, sauvegardes et observabilite. Les fichiers generes standard Angular/Flutter (wrappers, icones, plateformes Android/iOS, package-lock) sont produits par les CLI et volontairement omis du guide.

DDD pragmatique et limites

1. Decision

Tenant est le contexte d'identite organisationnelle; Catalog gere l'offre; Inventory gere la disponibilite; Order capture l'intention commerciale.

Critere d'acceptation: cette decision est visible dans le code, testee automatiquement ou suivie par une issue explicite avec responsable et echeance.

2. Decision

Les agregats ne traversent pas les bases. Une commande memorise le prix au moment de l'achat. Les identifiants UUID circulent dans les contrats.

Critere d'acceptation: cette decision est visible dans le code, testee automatiquement ou suivie par une issue explicite avec responsable et echeance.

3. Decision

Les use cases portent transactions et invariants; les controllers adaptent HTTP; les repositories adaptent la persistance.

Critere d'acceptation: cette decision est visible dans le code, testee automatiquement ou suivie par une issue explicite avec responsable et echeance.

4. Decision

Pour le MVP, la coherence stock/commande est eventuelle. Une evolution ajoute reservation, expiration et compensation explicites.

Critere d'acceptation: cette decision est visible dans le code, testee automatiquement ou suivie par une issue explicite avec responsable et echeance.

Isolation multi-tenant

1. Decision

Le claim `tenant_id` vient de Keycloak, jamais d'un header libre du client. `TenantFilter` initialise un `ThreadLocal` et le nettoie toujours dans `finally`.

Critere d'acceptation: cette decision est visible dans le code, testee automatiquement ou suivie par une issue explicite avec responsable et echeance.

2. Decision

Chaque requete repository inclut `tenant_id`. Les contraintes uniques incluent `tenant_id`. Les tests negatifs tentent l'acces croise.

Critere d'acceptation: cette decision est visible dans le code, testee automatiquement ou suivie par une issue explicite avec responsable et echeance.

3. Decision

Durcissement recommande: PostgreSQL Row Level Security avec `SET LOCAL app.tenant_id`, ou schema/base par tenant selon risque et cout.

Critere d'acceptation: cette decision est visible dans le code, testee automatiquement ou suivie par une issue explicite avec responsable et echeance.

4. Decision

Les logs incluent `tenant_id` pseudonymise, `subject` et `correlation-id`, sans token ni donnees sensibles.

Critere d'acceptation: cette decision est visible dans le code, testee automatiquement ou suivie par une issue explicite avec responsable et echeance.

Securite OAuth2/OIDC

1. Decision

Clients publics Angular et Flutter: Authorization Code + PKCE S256, aucun client_secret embarque.

Critere d'acceptation: cette decision est visible dans le code, testee automatiquement ou suivie par une issue explicite avec responsable et echeance.

2. Decision

Le gateway et les services valident issuer, signature, expiration et audience. Les autorisations combinent scopes/roles et tenant.

Critere d'acceptation: cette decision est visible dans le code, testee automatiquement ou suivie par une issue explicite avec responsable et echeance.

3. Decision

CORS limite les origines; CSRF est desactive uniquement pour API bearer stateless; TLS est obligatoire hors poste local.

Critere d'acceptation: cette decision est visible dans le code, testee automatiquement ou suivie par une issue explicite avec responsable et echeance.

4. Decision

Rotation de cles, MFA administrateur, sessions courtes, refresh token rotation et verrouillage brute-force sont configures dans Keycloak de production.

Critere d'acceptation: cette decision est visible dans le code, testee automatiquement ou suivie par une issue explicite avec responsable et echeance.

Outbox et Kafka

1. Decision

Commande et evenement sont inseres dans la meme transaction PostgreSQL. Le scheduler verrouille un lot SKIP LOCKED et marque published_at apres ack Kafka.

Critere d'acceptation: cette decision est visible dans le code, testee automatiquement ou suivie par une issue explicite avec responsable et echeance.

2. Decision

Le consommateur doit etre idempotent: ajouter processed_events(event_id, consumer) avec cle unique avant mise en production.

Critere d'acceptation: cette decision est visible dans le code, testee automatiquement ou suivie par une issue explicite avec responsable et echeance.

3. Decision

Le topic est versionne orders.created.v1; payload possede eventId, tenantId, orderId, productId et quantity.

Critere d'acceptation: cette decision est visible dans le code, testee automatiquement ou suivie par une issue explicite avec responsable et echeance.

4. Decision

DLQ, retries bornes, alertes de lag et nettoyage outbox sont indispensables. Debezium CDC peut remplacer le poller plus tard.

Critere d'acceptation: cette decision est visible dans le code, testee automatiquement ou suivie par une issue explicite avec responsable et echeance.

Strategie de tests

1. Decision

Tests unitaires: invariants, calcul total, absence de tenant. Tests slice MVC: validation et statuts. Tests integration: PostgreSQL reel via Testcontainers et migrations Flyway.

Critere d'acceptation: cette decision est visible dans le code, testee automatiquement ou suivie par une issue explicite avec responsable et echeance.

2. Decision

Tests de contrat valident JSON et compatibilite ascendante. Kafka Testcontainers verifie publication/consommation et idempotence.

Critere d'acceptation: cette decision est visible dans le code, testee automatiquement ou suivie par une issue explicite avec responsable et echeance.

3. Decision

Angular teste composants et services HTTP; Flutter teste parsing, etats loading/error et navigation. Playwright peut couvrir le parcours login-produit-stock-commande.

Critere d'acceptation: cette decision est visible dans le code, testee automatiquement ou suivie par une issue explicite avec responsable et echeance.

4. Decision

La CI bloque sur format, analyse statique, tests, build image, scan CVE et manifests valides.

Critere d'acceptation: cette decision est visible dans le code, testee automatiquement ou suivie par une issue explicite avec responsable et echeance.

DevOps cloud-agnostique

1. Decision

Les images sont non-root, immuables et taguées par SHA. Kubernetes se limite aux APIs portables; les variantes cloud passent par overlays Kustomize/Helm.

Critere d'acceptation: cette decision est visible dans le code, testee automatiquement ou suivie par une issue explicite avec responsable et echeance.

2. Decision

En production, PostgreSQL, Kafka et Keycloak sont preferablement manages ou operes par des operators matures avec sauvegardes testees.

Critere d'acceptation: cette decision est visible dans le code, testee automatiquement ou suivie par une issue explicite avec responsable et echeance.

3. Decision

Readiness indique si le pod accepte du trafic; liveness ne doit pas dependre d'un service externe. Requests/limits et PDB sont calibres par mesure.

Critere d'acceptation: cette decision est visible dans le code, testee automatiquement ou suivie par une issue explicite avec responsable et echeance.

4. Decision

GitOps applique manifests signes; External Secrets injecte les secrets; NetworkPolicies n'autorisent que les flux necessaires.

Critere d'acceptation: cette decision est visible dans le code, testee automatiquement ou suivie par une issue explicite avec responsable et echeance.

Runbook - Initialisation propre (Developpement)

Objectif

Creer le depot hors du projet medical; verifier git status; copier les fichiers; executer Maven, npm et Flutter avant le premier commit.

ProcEDURE exacte

1. Declarer l'incident ou le changement, le responsable, le tenant affecte et la fenetre. 2. Capturer les metriques avant action. 3. Executer d'abord en lecture seule. 4. Appliquer une seule modification reversible. 5. Verifier le walking skeleton complet. 6. Joindre preuves et decision au ticket.

```
git status --short; mvn clean verify; npm --prefix web ci; flutter test mobile
```

Controles de sortie

HTTP health vert; aucune augmentation des 5xx; latence dans le budget; aucune ligne cross-tenant; outbox sans age anormal; consumer lag stable; audit sans secret; rollback disponible.

Test a automatiser

Ajouter un test nomme initialisation_propre_regression couvrant le succes, l'echec, la reprise et l'isolation du tenant demo. En developpement, archiver resultat, version d'image et horodatage.

Runbook - Diagnostic login (Developpement)

Objectif

Comparer issuer public/interne, redirect URI exacte, heure systeme et claim tenant_id dans le JWT decode localement.

ProcEDURE exacte

1. Declarer l'incident ou le changement, le responsable, le tenant affecte et la fenetre. 2. Capturer les metriques avant action. 3. Executer d'abord en lecture seule. 4. Appliquer une seule modification reversible. 5. Verifier le walking skeleton complet. 6. Joindre preuves et decision au ticket.

```
curl http://localhost:8081/realms/b2b/.well-known/openid-configuration
```

Controles de sortie

HTTP health vert; aucune augmentation des 5xx; latence dans le budget; aucune ligne cross-tenant; outbox sans age anormal; consumer lag stable; audit sans secret; rollback disponible.

Test a automatiser

Ajouter un test nomme diagnostic_login_regression couvrant le succes, l'echec, la reprise et l'isolation du tenant demo. En developpement, archiver resultat, version d'image et horodatage.

Runbook - Diagnostic 401 (Developpement)

Objectif

Verifier Authorization Bearer, issuer, kid/JWKS, exp/nbf et audience. Ne jamais loguer le token complet.

Procedure exacte

1. Declarer l'incident ou le changement, le responsable, le tenant affecte et la fenetre. 2. Capturer les metriques avant action. 3. Executer d'abord en lecture seule. 4. Appliquer une seule modification reversible. 5. Verifier le walking skeleton complet. 6. Joindre preuves et decision au ticket.

```
curl -i http://localhost:8080/api/products
```

Controles de sortie

HTTP health vert; aucune augmentation des 5xx; latence dans le budget; aucune ligne cross-tenant; outbox sans age anormal; consumer lag stable; audit sans secret; rollback disponible.

Test a automatiser

Ajouter un test nomme diagnostic_401_regression couvrant le succes, l'echec, la reprise et l'isolation du tenant demo. En developpement, archiver resultat, version d'image et horodatage.

Runbook - Diagnostic 403 (Developpement)

Objectif

Verifier roles/scopes et mapping des authorities; distinguer authentication 401 et autorisation 403.

ProcEDURE exacte

1. Declarer l'incident ou le changement, le responsable, le tenant affecte et la fenetre. 2. Capturer les metriques avant action. 3. Executer d'abord en lecture seule. 4. Appliquer une seule modification reversible. 5. Verifier le walking skeleton complet. 6. Joindre preuves et decision au ticket.

```
kubect1 -n b2b logs deploy/gateway --since=10m
```

Controles de sortie

HTTP health vert; aucune augmentation des 5xx; latence dans le budget; aucune ligne cross-tenant; outbox sans age anormal; consumer lag stable; audit sans secret; rollback disponible.

Test a automatiser

Ajouter un test nomme diagnostic_403_regression couvrant le succes, l'echec, la reprise et l'isolation du tenant demo. En developpement, archiver resultat, version d'image et horodatage.

Runbook - Migration Flyway (Developpement)

Objectif

Sauvegarder, tester la migration sur clone, appliquer en compatibilite N/N-1, puis deployer le code.

ProcEDURE EXACTE

1. Declarer l'incident ou le changement, le responsable, le tenant affecte et la fenetre. 2. Capturer les metriques avant action. 3. Executer d'abord en lecture seule. 4. Appliquer une seule modification reversible. 5. Verifier le walking skeleton complet. 6. Joindre preuves et decision au ticket.

```
mvn -pl catalog-service flyway:info
```

Controles de sortie

HTTP health vert; aucune augmentation des 5xx; latence dans le budget; aucune ligne cross-tenant; outbox sans age anormal; consumer lag stable; audit sans secret; rollback disponible.

Test a automatiser

Ajouter un test nomme migration_flyway_regression couvrant le succes, l'echec, la reprise et l'isolation du tenant demo. En developpement, archiver resultat, version d'image et horodatage.

Runbook - Outbox bloquee (Developpement)

Objectif

Mesurer count/age des lignes pending, verifier Kafka, droits topic et erreurs du publisher avant toute relecture.

ProcEDURE exacte

1. Declarer l'incident ou le changement, le responsable, le tenant affecte et la fenetre. 2. Capturer les metriques avant action. 3. Executer d'abord en lecture seule. 4. Appliquer une seule modification reversible. 5. Verifier le walking skeleton complet. 6. Joindre preuves et decision au ticket.

```
select count(*),min(created_at) from outbox_events where published_at is null;
```

Controles de sortie

HTTP health vert; aucune augmentation des 5xx; latence dans le budget; aucune ligne cross-tenant; outbox sans age anormal; consumer lag stable; audit sans secret; rollback disponible.

Test a automatiser

Ajouter un test nomme outbox_bloquee_regression couvrant le succes, l'echec, la reprise et l'isolation du tenant demo. En developpement, archiver resultat, version d'image et horodatage.

Runbook - Lag consommateur (Developpement)

Objectif

Observer consumer group, partitions, rebalances et erreurs; scaler seulement si partitions suffisantes.

ProcEDURE exacte

1. Declarer l'incident ou le changement, le responsable, le tenant affecte et la fenetre. 2. Capturer les metriques avant action. 3. Executer d'abord en lecture seule. 4. Appliquer une seule modification reversible. 5. Verifier le walking skeleton complet. 6. Joindre preuves et decision au ticket.

```
docker compose exec kafka /opt/kafka/bin/kafka-consumer-groups.sh --bootstrap-server kafka:9092 --describe --group inventory
```

Controles de sortie

HTTP health vert; aucune augmentation des 5xx; latence dans le budget; aucune ligne cross-tenant; outbox sans age anormal; consumer lag stable; audit sans secret; rollback disponible.

Test a automatiser

Ajouter un test nomme lag_consommateur_regression couvrant le succes, l'echec, la reprise et l'isolation du tenant demo. En developpement, archiver resultat, version d'image et horodatage.

Runbook - Restauration PostgreSQL (Developpement)

Objectif

Tester regulierement restore dans un environnement isole et mesurer RPO/RTO, checksums et migrations.

ProcEDURE exacte

1. Declarer l'incident ou le changement, le responsable, le tenant affecte et la fenetre. 2. Capturer les metriques avant action. 3. Executer d'abord en lecture seule. 4. Appliquer une seule modification reversible. 5. Verifier le walking skeleton complet. 6. Joindre preuves et decision au ticket.

```
pg_dump -Fc -d catalog_db -f catalog.dump; createdb restore_test; pg_restore -d restore_test catalog.dump
```

Controles de sortie

HTTP health vert; aucune augmentation des 5xx; latence dans le budget; aucune ligne cross-tenant; outbox sans age anormal; consumer lag stable; audit sans secret; rollback disponible.

Test a automatiser

Ajouter un test nomme `restauration_postgresql_regression` couvrant le succes, l'echec, la reprise et l'isolation du tenant demo. En developpement, archiver resultat, version d'image et horodatage.

Runbook - Rollback applicatif (Developpement)

Objectif

Revenir a l'image precedente uniquement si schema backward-compatible; ne jamais down-migrer automatiquement des donnees.

ProcEDURE exacte

1. Declarer l'incident ou le changement, le responsable, le tenant affecte et la fenetre. 2. Capturer les metriques avant action. 3. Executer d'abord en lecture seule. 4. Appliquer une seule modification reversible. 5. Verifier le walking skeleton complet. 6. Joindre preuves et decision au ticket.

```
kubect1 -n b2b rollout undo deployment/catalog-service
```

Controles de sortie

HTTP health vert; aucune augmentation des 5xx; latence dans le budget; aucune ligne cross-tenant; outbox sans age anormal; consumer lag stable; audit sans secret; rollback disponible.

Test a automatiser

Ajouter un test nomme rollback_applicatif_regression couvrant le succes, l'echec, la reprise et l'isolation du tenant demo. En developpement, archiver resultat, version d'image et horodatage.

Runbook - Rotation secrets (Developpement)

Objectif

Creer nouvelle version, deployer les consommateurs, verifier, puis revoquer l'ancienne sans imprimer la valeur.

ProcEDURE exacte

1. Declarer l'incident ou le changement, le responsable, le tenant affecte et la fenetre. 2. Capturer les metriques avant action. 3. Executer d'abord en lecture seule. 4. Appliquer une seule modification reversible. 5. Verifier le walking skeleton complet. 6. Joindre preuves et decision au ticket.

```
kubect1 -n b2b rollout restart deployment/gateway
```

Controles de sortie

HTTP health vert; aucune augmentation des 5xx; latence dans le budget; aucune ligne cross-tenant; outbox sans age anormal; consumer lag stable; audit sans secret; rollback disponible.

Test a automatiser

Ajouter un test nomme rotation_secrets_regression couvrant le succes, l'echec, la reprise et l'isolation du tenant demo. En developpement, archiver resultat, version d'image et horodatage.

Runbook - SLO API (Developpement)

Objectif

Mesurer disponibilite, latence p95/p99 et taux d'erreur par route/tenant sans cardinalite non bornee.

Procedure exacte

1. Declarer l'incident ou le changement, le responsable, le tenant affecte et la fenetre. 2. Capturer les metriques avant action. 3. Executer d'abord en lecture seule. 4. Appliquer une seule modification reversible. 5. Verifier le walking skeleton complet. 6. Joindre preuves et decision au ticket.

```
kubect1 -n b2b top pods
```

Controles de sortie

HTTP health vert; aucune augmentation des 5xx; latence dans le budget; aucune ligne cross-tenant; outbox sans age anormal; consumer lag stable; audit sans secret; rollback disponible.

Test a automatiser

Ajouter un test nomme slo_api_regression couvrant le succes, l'echec, la reprise et l'isolation du tenant demo. En developpement, archiver resultat, version d'image et horodatage.

Runbook - Test de charge (Developpement)

Objectif

Injecter catalogue et commandes realistes; verifier pool JDBC, Kafka lag, CPU, memoire et contention SQL.

ProcEDURE EXACTE

1. Declarer l'incident ou le changement, le responsable, le tenant affecte et la fenetre. 2. Capturer les metriques avant action. 3. Executer d'abord en lecture seule. 4. Appliquer une seule modification reversible. 5. Verifier le walking skeleton complet. 6. Joindre preuves et decision au ticket.

```
k6 run smoke.js
```

Controles de sortie

HTTP health vert; aucune augmentation des 5xx; latence dans le budget; aucune ligne cross-tenant; outbox sans age anormal; consumer lag stable; audit sans secret; rollback disponible.

Test a automatiser

Ajouter un test nomme test_de_charge_regression couvrant le succes, l'echec, la reprise et l'isolation du tenant demo. En developpement, archiver resultat, version d'image et horodatage.

Runbook - Incident isolation tenant (Developpement)

Objectif

Stopper fuite, conserver preuves, identifier tenants/objets, corriger predicate et ajouter test de regression.

ProcEDURE exacte

1. Declarer l'incident ou le changement, le responsable, le tenant affecte et la fenetre. 2. Capturer les metriques avant action. 3. Executer d'abord en lecture seule. 4. Appliquer une seule modification reversible. 5. Verifier le walking skeleton complet. 6. Joindre preuves et decision au ticket.

```
select tenant_id,count(*) from products group by tenant_id;
```

Controles de sortie

HTTP health vert; aucune augmentation des 5xx; latence dans le budget; aucune ligne cross-tenant; outbox sans age anormal; consumer lag stable; audit sans secret; rollback disponible.

Test a automatiser

Ajouter un test nomme incident_isolation_tenant_regression couvrant le succes, l'echec, la reprise et l'isolation du tenant demo. En developpement, archiver resultat, version d'image et horodatage.

Runbook - Validation Kubernetes (Developpement)

Objectif

Valider schema, policy, images non-root, resources, probes et absence de secret en clair.

Procédure exacte

1. Declarer l'incident ou le changement, le responsable, le tenant affecte et la fenetre. 2. Capturer les metriques avant action. 3. Executer d'abord en lecture seule. 4. Appliquer une seule modification reversible. 5. Verifier le walking skeleton complet. 6. Joindre preuves et decision au ticket.

```
kubect1 apply --dry-run=server -f infra/k8s/
```

Controles de sortie

HTTP health vert; aucune augmentation des 5xx; latence dans le budget; aucune ligne cross-tenant; outbox sans age anormal; consumer lag stable; audit sans secret; rollback disponible.

Test a automatiser

Ajouter un test nomme validation_kubernetes_regression couvrant le succes, l'echec, la reprise et l'isolation du tenant demo. En developpement, archiver resultat, version d'image et horodatage.

Runbook - Build reproductible (Developpement)

Objectif

Utiliser lockfiles, versions d'images par digest, Maven dependency tree et SBOM archivee.

ProcEDURE EXACTE

1. Declarer l'incident ou le changement, le responsable, le tenant affecte et la fenetre. 2. Capturer les metriques avant action. 3. Executer d'abord en lecture seule. 4. Appliquer une seule modification reversible. 5. Verifier le walking skeleton complet. 6. Joindre preuves et decision au ticket.

```
mvn -DskipTests dependency:tree; npm --prefix web ci
```

Controles de sortie

HTTP health vert; aucune augmentation des 5xx; latence dans le budget; aucune ligne cross-tenant; outbox sans age anormal; consumer lag stable; audit sans secret; rollback disponible.

Test a automatiser

Ajouter un test nomme build_reproductible_regression couvrant le succes, l'echec, la reprise et l'isolation du tenant demo. En developpement, archiver resultat, version d'image et horodatage.

Runbook - Release canary (Developpement)

Objectif

Deployer faible trafic, comparer erreurs/latence/Kafka lag, promouvoir ou rollback selon seuil predefini.

ProcEDURE exacte

1. Declarer l'incident ou le changement, le responsable, le tenant affecte et la fenetre. 2. Capturer les metriques avant action. 3. Executer d'abord en lecture seule. 4. Appliquer une seule modification reversible. 5. Verifier le walking skeleton complet. 6. Joindre preuves et decision au ticket.

```
kubect1 -n b2b rollout status deployment/order-service
```

Controles de sortie

HTTP health vert; aucune augmentation des 5xx; latence dans le budget; aucune ligne cross-tenant; outbox sans age anormal; consumer lag stable; audit sans secret; rollback disponible.

Test a automatiser

Ajouter un test nomme release_canary_regression couvrant le succes, l'echec, la reprise et l'isolation du tenant demo. En developpement, archiver resultat, version d'image et horodatage.

Runbook - Observabilite trace (Developpement)

Objectif

Propager traceparent et correlation-id HTTP vers messages Kafka; relier commande et evenement sans PII.

Procedure exacte

1. Declarer l'incident ou le changement, le responsable, le tenant affecte et la fenetre. 2. Capturer les metriques avant action. 3. Executer d'abord en lecture seule. 4. Appliquer une seule modification reversible. 5. Verifier le walking skeleton complet. 6. Joindre preuves et decision au ticket.

```
curl -H 'traceparent: 00-00000000000000000000000000000001-0000000000000001-01' ...
```

Controles de sortie

HTTP health vert; aucune augmentation des 5xx; latence dans le budget; aucune ligne cross-tenant; outbox sans age anormal; consumer lag stable; audit sans secret; rollback disponible.

Test a automatiser

Ajouter un test nomme observabilite_trace_regression couvrant le succes, l'echec, la reprise et l'isolation du tenant demo. En developpement, archiver resultat, version d'image et horodatage.

Runbook - Nettoyage outbox (Developpement)

Objectif

Supprimer par lots les evenements publies au-dela de retention, apres verification audit et replay.

ProcEDURE EXACTE

1. Declarer l'incident ou le changement, le responsable, le tenant affecte et la fenetre. 2. Capturer les metriques avant action. 3. Executer d'abord en lecture seule. 4. Appliquer une seule modification reversible. 5. Verifier le walking skeleton complet. 6. Joindre preuves et decision au ticket.

```
delete from outbox_events where id in (select id from outbox_events where published_at < now()-interval '30 days'
limit 1000);
```

Controles de sortie

HTTP health vert; aucune augmentation des 5xx; latence dans le budget; aucune ligne cross-tenant; outbox sans age anormal; consumer lag stable; audit sans secret; rollback disponible.

Test a automatiser

Ajouter un test nomme nettoyage_outbox_regression couvrant le succes, l'echec, la reprise et l'isolation du tenant demo. En developpement, archiver resultat, version d'image et horodatage.

Runbook - DR exercice (Developpement)

Objectif

Simuler perte region/cluster, restaurer donnees, DNS et cles; documenter temps reel et ecarts.

ProcEDURE exacte

1. Declarer l'incident ou le changement, le responsable, le tenant affecte et la fenetre. 2. Capturer les metriques avant action. 3. Executer d'abord en lecture seule. 4. Appliquer une seule modification reversible. 5. Verifier le walking skeleton complet. 6. Joindre preuves et decision au ticket.

```
kind delete cluster --name b2b-mvp; ./scripts/kind-up.ps1
```

Controles de sortie

HTTP health vert; aucune augmentation des 5xx; latence dans le budget; aucune ligne cross-tenant; outbox sans age anormal; consumer lag stable; audit sans secret; rollback disponible.

Test a automatiser

Ajouter un test nomme dr_exercice_regression couvrant le succes, l'echec, la reprise et l'isolation du tenant demo. En developpement, archiver resultat, version d'image et horodatage.

Runbook - Revue avant production (Developpement)

Objectif

Supprimer comptes demo, mots de passe locaux et start-dev; activer TLS, HA, sauvegardes, politiques et alertes.

ProcEDURE exacte

1. Declarer l'incident ou le changement, le responsable, le tenant affecte et la fenetre. 2. Capturer les metriques avant action. 3. Executer d'abord en lecture seule. 4. Appliquer une seule modification reversible. 5. Verifier le walking skeleton complet. 6. Joindre preuves et decision au ticket.

```
rg 'local-only-change-me|start-dev|buyer@demo' .
```

Controles de sortie

HTTP health vert; aucune augmentation des 5xx; latence dans le budget; aucune ligne cross-tenant; outbox sans age anormal; consumer lag stable; audit sans secret; rollback disponible.

Test a automatiser

Ajouter un test nomme revue_avant_production_regression couvrant le succes, l'echec, la reprise et l'isolation du tenant demo. En developpement, archiver resultat, version d'image et horodatage.

Runbook - Initialisation propre (Integration continue)

Objectif

Creer le depot hors du projet medical; verifier git status; copier les fichiers; executer Maven, npm et Flutter avant le premier commit.

ProcEDURE EXACTE

1. Declarer l'incident ou le changement, le responsable, le tenant affecte et la fenetre. 2. Capturer les metriques avant action. 3. Executer d'abord en lecture seule. 4. Appliquer une seule modification reversible. 5. Verifier le walking skeleton complet. 6. Joindre preuves et decision au ticket.

```
git status --short; mvn clean verify; npm --prefix web ci; flutter test mobile
```

Controles de sortie

HTTP health vert; aucune augmentation des 5xx; latence dans le budget; aucune ligne cross-tenant; outbox sans age anormal; consumer lag stable; audit sans secret; rollback disponible.

Test a automatiser

Ajouter un test nomme initialisation_propre_regression couvrant le succes, l'echec, la reprise et l'isolation du tenant demo. En integration continue, archiver resultat, version d'image et horodatage.

Runbook - Diagnostic login (Integration continue)

Objectif

Comparer issuer public/interne, redirect URI exacte, heure systeme et claim tenant_id dans le JWT decode localement.

ProcEDURE exacte

1. Declarer l'incident ou le changement, le responsable, le tenant affecte et la fenetre. 2. Capturer les metriques avant action. 3. Executer d'abord en lecture seule. 4. Appliquer une seule modification reversible. 5. Verifier le walking skeleton complet. 6. Joindre preuves et decision au ticket.

```
curl http://localhost:8081/realms/b2b/.well-known/openid-configuration
```

Controles de sortie

HTTP health vert; aucune augmentation des 5xx; latence dans le budget; aucune ligne cross-tenant; outbox sans age anormal; consumer lag stable; audit sans secret; rollback disponible.

Test a automatiser

Ajouter un test nomme diagnostic_login_regression couvrant le succes, l'echec, la reprise et l'isolation du tenant demo. En integration continue, archiver resultat, version d'image et horodatage.

Runbook - Diagnostic 401 (Integration continue)

Objectif

Verifier Authorization Bearer, issuer, kid/JWKS, exp/nbf et audience. Ne jamais loguer le token complet.

Procédure exacte

1. Declarer l'incident ou le changement, le responsable, le tenant affecte et la fenetre. 2. Capturer les metriques avant action. 3. Executer d'abord en lecture seule. 4. Appliquer une seule modification reversible. 5. Verifier le walking skeleton complet. 6. Joindre preuves et decision au ticket.

```
curl -i http://localhost:8080/api/products
```

Controles de sortie

HTTP health vert; aucune augmentation des 5xx; latence dans le budget; aucune ligne cross-tenant; outbox sans age anormal; consumer lag stable; audit sans secret; rollback disponible.

Test a automatiser

Ajouter un test nomme diagnostic_401_regression couvrant le succes, l'echec, la reprise et l'isolation du tenant demo. En integration continue, archiver resultat, version d'image et horodatage.

Runbook - Diagnostic 403 (Integration continue)

Objectif

Verifier roles/scopes et mapping des authorities; distinguer authentication 401 et autorisation 403.

Procédure exacte

1. Declarer l'incident ou le changement, le responsable, le tenant affecte et la fenetre. 2. Capturer les metriques avant action. 3. Executer d'abord en lecture seule. 4. Appliquer une seule modification reversible. 5. Verifier le walking skeleton complet. 6. Joindre preuves et decision au ticket.

```
kubect1 -n b2b logs deploy/gateway --since=10m
```

Contrôles de sortie

HTTP health vert; aucune augmentation des 5xx; latence dans le budget; aucune ligne cross-tenant; outbox sans age anormal; consumer lag stable; audit sans secret; rollback disponible.

Test a automatiser

Ajouter un test nomme diagnostic_403_regression couvrant le succes, l'echec, la reprise et l'isolation du tenant demo. En integration continue, archiver resultat, version d'image et horodatage.

Runbook - Migration Flyway (Integration continue)

Objectif

Sauvegarder, tester la migration sur clone, appliquer en compatibilite N/N-1, puis deployer le code.

ProcEDURE EXACTE

1. Declarer l'incident ou le changement, le responsable, le tenant affecte et la fenetre. 2. Capturer les metriques avant action. 3. Executer d'abord en lecture seule. 4. Appliquer une seule modification reversible. 5. Verifier le walking skeleton complet. 6. Joindre preuves et decision au ticket.

```
mvn -pl catalog-service flyway:info
```

Controles de sortie

HTTP health vert; aucune augmentation des 5xx; latence dans le budget; aucune ligne cross-tenant; outbox sans age anormal; consumer lag stable; audit sans secret; rollback disponible.

Test a automatiser

Ajouter un test nomme migration_flyway_regression couvrant le succes, l'echec, la reprise et l'isolation du tenant demo. En integration continue, archiver resultat, version d'image et horodatage.

Runbook - Outbox bloquee (Integration continue)

Objectif

Mesurer count/age des lignes pending, verifier Kafka, droits topic et erreurs du publisher avant toute relecture.

ProcEDURE EXACTE

1. Declarer l'incident ou le changement, le responsable, le tenant affecte et la fenetre. 2. Capturer les metriques avant action. 3. Executer d'abord en lecture seule. 4. Appliquer une seule modification reversible. 5. Verifier le walking skeleton complet. 6. Joindre preuves et decision au ticket.

```
select count(*),min(created_at) from outbox_events where published_at is null;
```

Controles de sortie

HTTP health vert; aucune augmentation des 5xx; latence dans le budget; aucune ligne cross-tenant; outbox sans age anormal; consumer lag stable; audit sans secret; rollback disponible.

Test a automatiser

Ajouter un test nomme outbox_bloquee_regression couvrant le succes, l'echec, la reprise et l'isolation du tenant demo. En integration continue, archiver resultat, version d'image et horodatage.

Runbook - Lag consommateur (Integration continue)

Objectif

Observer consumer group, partitions, rebalances et erreurs; scaler seulement si partitions suffisantes.

Procédure exacte

1. Declarer l'incident ou le changement, le responsable, le tenant affecte et la fenetre. 2. Capturer les metriques avant action. 3. Executer d'abord en lecture seule. 4. Appliquer une seule modification reversible. 5. Verifier le walking skeleton complet. 6. Joindre preuves et decision au ticket.

```
docker compose exec kafka /opt/kafka/bin/kafka-consumer-groups.sh --bootstrap-server kafka:9092 --describe --group inventory
```

Controles de sortie

HTTP health vert; aucune augmentation des 5xx; latence dans le budget; aucune ligne cross-tenant; outbox sans age anormal; consumer lag stable; audit sans secret; rollback disponible.

Test a automatiser

Ajouter un test nomme lag_consommateur_regression couvrant le succes, l'echec, la reprise et l'isolation du tenant demo. En integration continue, archiver resultat, version d'image et horodatage.

Runbook - Restauration PostgreSQL (Integration continue)

Objectif

Tester regulierement restore dans un environnement isole et mesurer RPO/RTO, checksums et migrations.

ProcEDURE exacte

1. Declarer l'incident ou le changement, le responsable, le tenant affecte et la fenetre. 2. Capturer les metriques avant action. 3. Executer d'abord en lecture seule. 4. Appliquer une seule modification reversible. 5. Verifier le walking skeleton complet. 6. Joindre preuves et decision au ticket.

```
pg_dump -Fc -d catalog_db -f catalog.dump; createdb restore_test; pg_restore -d restore_test catalog.dump
```

Controles de sortie

HTTP health vert; aucune augmentation des 5xx; latence dans le budget; aucune ligne cross-tenant; outbox sans age anormal; consumer lag stable; audit sans secret; rollback disponible.

Test a automatiser

Ajouter un test nomme `restauration_postgresql_regression` couvrant le succes, l'echec, la reprise et l'isolation du tenant demo. En integration continue, archiver resultat, version d'image et horodatage.

Runbook - Rollback applicatif (Integration continue)

Objectif

Revenir a l'image precedente uniquement si schema backward-compatible; ne jamais down-migrer automatiquement des donnees.

ProcEDURE exacte

1. Declarer l'incident ou le changement, le responsable, le tenant affecte et la fenetre. 2. Capturer les metriques avant action. 3. Executer d'abord en lecture seule. 4. Appliquer une seule modification reversible. 5. Verifier le walking skeleton complet. 6. Joindre preuves et decision au ticket.

```
kubect1 -n b2b rollout undo deployment/catalog-service
```

Controles de sortie

HTTP health vert; aucune augmentation des 5xx; latence dans le budget; aucune ligne cross-tenant; outbox sans age anormal; consumer lag stable; audit sans secret; rollback disponible.

Test a automatiser

Ajouter un test nomme rollback_applicatif_regression couvrant le succes, l'echec, la reprise et l'isolation du tenant demo. En integration continue, archiver resultat, version d'image et horodatage.

Runbook - Rotation secrets (Integration continue)

Objectif

Creer nouvelle version, deployer les consommateurs, verifier, puis revoquer l'ancienne sans imprimer la valeur.

ProcEDURE EXACTE

1. Declarer l'incident ou le changement, le responsable, le tenant affecte et la fenetre. 2. Capturer les metriques avant action. 3. Executer d'abord en lecture seule. 4. Appliquer une seule modification reversible. 5. Verifier le walking skeleton complet. 6. Joindre preuves et decision au ticket.

```
kubect1 -n b2b rollout restart deployment/gateway
```

Controles de sortie

HTTP health vert; aucune augmentation des 5xx; latence dans le budget; aucune ligne cross-tenant; outbox sans age anormal; consumer lag stable; audit sans secret; rollback disponible.

Test a automatiser

Ajouter un test nomme rotation_secrets_regression couvrant le succes, l'echec, la reprise et l'isolation du tenant demo. En integration continue, archiver resultat, version d'image et horodatage.

Runbook - SLO API (Integration continue)

Objectif

Mesurer disponibilite, latence p95/p99 et taux d'erreur par route/tenant sans cardinalite non bornee.

Procedure exacte

1. Declarer l'incident ou le changement, le responsable, le tenant affecte et la fenetre. 2. Capturer les metriques avant action. 3. Executer d'abord en lecture seule. 4. Appliquer une seule modification reversible. 5. Verifier le walking skeleton complet. 6. Joindre preuves et decision au ticket.

```
kubect1 -n b2b top pods
```

Controles de sortie

HTTP health vert; aucune augmentation des 5xx; latence dans le budget; aucune ligne cross-tenant; outbox sans age anormal; consumer lag stable; audit sans secret; rollback disponible.

Test a automatiser

Ajouter un test nomme slo_api_regression couvrant le succes, l'echec, la reprise et l'isolation du tenant demo. En integration continue, archiver resultat, version d'image et horodatage.

Runbook - Test de charge (Integration continue)

Objectif

Injecter catalogue et commandes realistes; verifier pool JDBC, Kafka lag, CPU, memoire et contention SQL.

ProcEDURE EXACTE

1. Declarer l'incident ou le changement, le responsable, le tenant affecte et la fenetre. 2. Capturer les metriques avant action. 3. Executer d'abord en lecture seule. 4. Appliquer une seule modification reversible. 5. Verifier le walking skeleton complet. 6. Joindre preuves et decision au ticket.

```
k6 run smoke.js
```

Controles de sortie

HTTP health vert; aucune augmentation des 5xx; latence dans le budget; aucune ligne cross-tenant; outbox sans age anormal; consumer lag stable; audit sans secret; rollback disponible.

Test a automatiser

Ajouter un test nomme test_de_charge_regression couvrant le succes, l'echec, la reprise et l'isolation du tenant demo. En integration continue, archiver resultat, version d'image et horodatage.

Runbook - Incident isolation tenant (Integration continue)

Objectif

Stopper fuite, conserver preuves, identifier tenants/objets, corriger predicate et ajouter test de regression.

Procédure exacte

1. Déclarer l'incident ou le changement, le responsable, le tenant affecté et la fenêtre. 2. Capturer les métriques avant action. 3. Exécuter d'abord en lecture seule. 4. Appliquer une seule modification réversible. 5. Vérifier le walking skeleton complet. 6. Joindre preuves et décision au ticket.

```
select tenant_id,count(*) from products group by tenant_id;
```

Contrôles de sortie

HTTP health vert; aucune augmentation des 5xx; latence dans le budget; aucune ligne cross-tenant; outbox sans âge anormal; consumer lag stable; audit sans secret; rollback disponible.

Test à automatiser

Ajouter un test nommé incident_isolation_tenant_regression couvrant le succès, l'échec, la reprise et l'isolation du tenant demo. En intégration continue, archiver résultat, version d'image et horodatage.

Runbook - Validation Kubernetes (Integration continue)

Objectif

Valider schema, policy, images non-root, resources, probes et absence de secret en clair.

Procédure exacte

1. Declarer l'incident ou le changement, le responsable, le tenant affecte et la fenetre. 2. Capturer les metriques avant action. 3. Executer d'abord en lecture seule. 4. Appliquer une seule modification reversible. 5. Verifier le walking skeleton complet. 6. Joindre preuves et decision au ticket.

```
kubectl apply --dry-run=server -f infra/k8s/
```

Controles de sortie

HTTP health vert; aucune augmentation des 5xx; latence dans le budget; aucune ligne cross-tenant; outbox sans age anormal; consumer lag stable; audit sans secret; rollback disponible.

Test a automatiser

Ajouter un test nomme validation_kubernetes_regression couvrant le succes, l'echec, la reprise et l'isolation du tenant demo. En integration continue, archiver resultat, version d'image et horodatage.

Runbook - Build reproductible (Integration continue)

Objectif

Utiliser lockfiles, versions d'images par digest, Maven dependency tree et SBOM archivee.

Procédure exacte

1. Declarer l'incident ou le changement, le responsable, le tenant affecte et la fenetre. 2. Capturer les metriques avant action. 3. Executer d'abord en lecture seule. 4. Appliquer une seule modification reversible. 5. Verifier le walking skeleton complet. 6. Joindre preuves et decision au ticket.

```
mvn -DskipTests dependency:tree; npm --prefix web ci
```

Controles de sortie

HTTP health vert; aucune augmentation des 5xx; latence dans le budget; aucune ligne cross-tenant; outbox sans age anormal; consumer lag stable; audit sans secret; rollback disponible.

Test a automatiser

Ajouter un test nomme build_reproductible_regression couvrant le succes, l'echec, la reprise et l'isolation du tenant demo. En integration continue, archiver resultat, version d'image et horodatage.

Runbook - Release canary (Integration continue)

Objectif

Deployer faible trafic, comparer erreurs/latence/Kafka lag, promouvoir ou rollback selon seuil predefini.

ProcEDURE EXACTE

1. Declarer l'incident ou le changement, le responsable, le tenant affecte et la fenetre. 2. Capturer les metriques avant action. 3. Executer d'abord en lecture seule. 4. Appliquer une seule modification reversible. 5. Verifier le walking skeleton complet. 6. Joindre preuves et decision au ticket.

```
kubect1 -n b2b rollout status deployment/order-service
```

Controles de sortie

HTTP health vert; aucune augmentation des 5xx; latence dans le budget; aucune ligne cross-tenant; outbox sans age anormal; consumer lag stable; audit sans secret; rollback disponible.

Test a automatiser

Ajouter un test nomme release_canary_regression couvrant le succes, l'echec, la reprise et l'isolation du tenant demo. En integration continue, archiver resultat, version d'image et horodatage.

Runbook - Observabilite trace (Integration continue)

Objectif

Propager traceparent et correlation-id HTTP vers messages Kafka; relier commande et evenement sans PII.

Procedure exacte

1. Declarer l'incident ou le changement, le responsable, le tenant affecte et la fenetre. 2. Capturer les metriques avant action. 3. Executer d'abord en lecture seule. 4. Appliquer une seule modification reversible. 5. Verifier le walking skeleton complet. 6. Joindre preuves et decision au ticket.

```
curl -H 'traceparent: 00-00000000000000000000000000000001-0000000000000001-01' ...
```

Controles de sortie

HTTP health vert; aucune augmentation des 5xx; latence dans le budget; aucune ligne cross-tenant; outbox sans age anormal; consumer lag stable; audit sans secret; rollback disponible.

Test a automatiser

Ajouter un test nomme observabilite_trace_regression couvrant le succes, l'echec, la reprise et l'isolation du tenant demo. En integration continue, archiver resultat, version d'image et horodatage.

Runbook - Nettoyage outbox (Integration continue)

Objectif

Supprimer par lots les evenements publies au-dela de retention, apres verification audit et replay.

ProcEDURE EXACTE

1. Declarer l'incident ou le changement, le responsable, le tenant affecte et la fenetre. 2. Capturer les metriques avant action. 3. Executer d'abord en lecture seule. 4. Appliquer une seule modification reversible. 5. Verifier le walking skeleton complet. 6. Joindre preuves et decision au ticket.

```
delete from outbox_events where id in (select id from outbox_events where published_at < now()-interval '30 days'
limit 1000);
```

Controles de sortie

HTTP health vert; aucune augmentation des 5xx; latence dans le budget; aucune ligne cross-tenant; outbox sans age anormal; consumer lag stable; audit sans secret; rollback disponible.

Test a automatiser

Ajouter un test nomme nettoyage_outbox_regression couvrant le succes, l'echec, la reprise et l'isolation du tenant demo. En integration continue, archiver resultat, version d'image et horodatage.

Runbook - DR exercice (Integration continue)

Objectif

Simuler perte region/cluster, restaurer donnees, DNS et cles; documenter temps reel et ecarts.

ProcEDURE EXACTE

1. Declarer l'incident ou le changement, le responsable, le tenant affecte et la fenetre. 2. Capturer les metriques avant action. 3. Executer d'abord en lecture seule. 4. Appliquer une seule modification reversible. 5. Verifier le walking skeleton complet. 6. Joindre preuves et decision au ticket.

```
kind delete cluster --name b2b-mvp; ./scripts/kind-up.ps1
```

Controles de sortie

HTTP health vert; aucune augmentation des 5xx; latence dans le budget; aucune ligne cross-tenant; outbox sans age anormal; consumer lag stable; audit sans secret; rollback disponible.

Test a automatiser

Ajouter un test nomme dr_exercice_regression couvrant le succes, l'echec, la reprise et l'isolation du tenant demo. En integration continue, archiver resultat, version d'image et horodatage.

Runbook - Revue avant production (Integration continue)

Objectif

Supprimer comptes demo, mots de passe locaux et start-dev; activer TLS, HA, sauvegardes, politiques et alertes.

ProcEDURE EXACTE

1. Declarer l'incident ou le changement, le responsable, le tenant affecte et la fenetre. 2. Capturer les metriques avant action. 3. Executer d'abord en lecture seule. 4. Appliquer une seule modification reversible. 5. Verifier le walking skeleton complet. 6. Joindre preuves et decision au ticket.

```
rg 'local-only-change-me|start-dev|buyer@demo' .
```

Controles de sortie

HTTP health vert; aucune augmentation des 5xx; latence dans le budget; aucune ligne cross-tenant; outbox sans age anormal; consumer lag stable; audit sans secret; rollback disponible.

Test a automatiser

Ajouter un test nomme revue_avant_production_regression couvrant le succes, l'echec, la reprise et l'isolation du tenant demo. En integration continue, archiver resultat, version d'image et horodatage.

Runbook - Initialisation propre (Preproduction)

Objectif

Creer le depot hors du projet medical; verifier git status; copier les fichiers; executer Maven, npm et Flutter avant le premier commit.

ProcEDURE EXACTE

1. Declarer l'incident ou le changement, le responsable, le tenant affecte et la fenetre. 2. Capturer les metriques avant action. 3. Executer d'abord en lecture seule. 4. Appliquer une seule modification reversible. 5. Verifier le walking skeleton complet. 6. Joindre preuves et decision au ticket.

```
git status --short; mvn clean verify; npm --prefix web ci; flutter test mobile
```

Controles de sortie

HTTP health vert; aucune augmentation des 5xx; latence dans le budget; aucune ligne cross-tenant; outbox sans age anormal; consumer lag stable; audit sans secret; rollback disponible.

Test a automatiser

Ajouter un test nomme initialisation_propre_regression couvrant le succes, l'echec, la reprise et l'isolation du tenant demo. En preproduction, archiver resultat, version d'image et horodatage.

Runbook - Diagnostic login (Preproduction)

Objectif

Comparer issuer public/interne, redirect URI exacte, heure systeme et claim tenant_id dans le JWT decode localement.

ProcEDURE exacte

1. Declarer l'incident ou le changement, le responsable, le tenant affecte et la fenetre. 2. Capturer les metriques avant action. 3. Executer d'abord en lecture seule. 4. Appliquer une seule modification reversible. 5. Verifier le walking skeleton complet. 6. Joindre preuves et decision au ticket.

```
curl http://localhost:8081/realms/b2b/.well-known/openid-configuration
```

Controles de sortie

HTTP health vert; aucune augmentation des 5xx; latence dans le budget; aucune ligne cross-tenant; outbox sans age anormal; consumer lag stable; audit sans secret; rollback disponible.

Test a automatiser

Ajouter un test nomme diagnostic_login_regression couvrant le succes, l'echec, la reprise et l'isolation du tenant demo. En preproduction, archiver resultat, version d'image et horodatage.

Runbook - Diagnostic 401 (Preproduction)

Objectif

Vérifier Authorization Bearer, issuer, kid/JWKS, exp/nbf et audience. Ne jamais logger le token complet.

Procédure exacte

1. Déclarer l'incident ou le changement, le responsable, le tenant affecté et la fenêtre. 2. Capturer les métriques avant action. 3. Exécuter d'abord en lecture seule. 4. Appliquer une seule modification réversible. 5. Vérifier le walking skeleton complet. 6. Joindre preuves et décision au ticket.

```
curl -i http://localhost:8080/api/products
```

Contrôles de sortie

HTTP health vert; aucune augmentation des 5xx; latence dans le budget; aucune ligne cross-tenant; outbox sans âge anormal; consumer lag stable; audit sans secret; rollback disponible.

Test à automatiser

Ajouter un test nommé diagnostic_401_regression couvrant le succès, l'échec, la reprise et l'isolation du tenant demo. En préproduction, archiver résultat, version d'image et horodatage.

Runbook - Diagnostic 403 (Preproduction)

Objectif

Verifier roles/scopes et mapping des authorities; distinguer authentication 401 et autorisation 403.

Procédure exacte

1. Declarer l'incident ou le changement, le responsable, le tenant affecte et la fenetre. 2. Capturer les metriques avant action. 3. Executer d'abord en lecture seule. 4. Appliquer une seule modification reversible. 5. Verifier le walking skeleton complet. 6. Joindre preuves et decision au ticket.

```
kubectl -n b2b logs deploy/gateway --since=10m
```

Contrôles de sortie

HTTP health vert; aucune augmentation des 5xx; latence dans le budget; aucune ligne cross-tenant; outbox sans age anormal; consumer lag stable; audit sans secret; rollback disponible.

Test a automatiser

Ajouter un test nomme diagnostic_403_regression couvrant le succes, l'echec, la reprise et l'isolation du tenant demo. En preproduction, archiver resultat, version d'image et horodatage.

Runbook - Migration Flyway (Preproduction)

Objectif

Sauvegarder, tester la migration sur clone, appliquer en compatibilite N/N-1, puis deployer le code.

ProcEDURE EXACTE

1. Declarer l'incident ou le changement, le responsable, le tenant affecte et la fenetre. 2. Capturer les metriques avant action. 3. Executer d'abord en lecture seule. 4. Appliquer une seule modification reversible. 5. Verifier le walking skeleton complet. 6. Joindre preuves et decision au ticket.

```
mvn -pl catalog-service flyway:info
```

Controles de sortie

HTTP health vert; aucune augmentation des 5xx; latence dans le budget; aucune ligne cross-tenant; outbox sans age anormal; consumer lag stable; audit sans secret; rollback disponible.

Test a automatiser

Ajouter un test nomme migration_flyway_regression couvrant le succes, l'echec, la reprise et l'isolation du tenant demo. En preproduction, archiver resultat, version d'image et horodatage.

Runbook - Outbox bloquee (Preproduction)

Objectif

Mesurer count/age des lignes pending, verifier Kafka, droits topic et erreurs du publisher avant toute relecture.

ProcEDURE exacte

1. Declarer l'incident ou le changement, le responsable, le tenant affecte et la fenetre. 2. Capturer les metriques avant action. 3. Executer d'abord en lecture seule. 4. Appliquer une seule modification reversible. 5. Verifier le walking skeleton complet. 6. Joindre preuves et decision au ticket.

```
select count(*),min(created_at) from outbox_events where published_at is null;
```

Controles de sortie

HTTP health vert; aucune augmentation des 5xx; latence dans le budget; aucune ligne cross-tenant; outbox sans age anormal; consumer lag stable; audit sans secret; rollback disponible.

Test a automatiser

Ajouter un test nomme outbox_bloquee_regression couvrant le succes, l'echec, la reprise et l'isolation du tenant demo. En preproduction, archiver resultat, version d'image et horodatage.

Runbook - Lag consommateur (Preproduction)

Objectif

Observer consumer group, partitions, rebalances et erreurs; scaler seulement si partitions suffisantes.

Procédure exacte

1. Déclarer l'incident ou le changement, le responsable, le tenant affecté et la fenêtre. 2. Capturer les métriques avant action. 3. Exécuter d'abord en lecture seule. 4. Appliquer une seule modification réversible. 5. Vérifier le walking skeleton complet. 6. Joindre preuves et décision au ticket.

```
docker compose exec kafka /opt/kafka/bin/kafka-consumer-groups.sh --bootstrap-server kafka:9092 --describe --group inventory
```

Contrôles de sortie

HTTP health vert; aucune augmentation des 5xx; latence dans le budget; aucune ligne cross-tenant; outbox sans âge anormal; consumer lag stable; audit sans secret; rollback disponible.

Test à automatiser

Ajouter un test nommé lag_consommateur_regression couvrant le succès, l'échec, la reprise et l'isolation du tenant demo. En préproduction, archiver résultat, version d'image et horodatage.

Runbook - Restauration PostgreSQL (Preproduction)

Objectif

Tester regulierement restore dans un environnement isole et mesurer RPO/RTO, checksums et migrations.

ProcEDURE exacte

1. Declarer l'incident ou le changement, le responsable, le tenant affecte et la fenetre. 2. Capturer les metriques avant action. 3. Executer d'abord en lecture seule. 4. Appliquer une seule modification reversible. 5. Verifier le walking skeleton complet. 6. Joindre preuves et decision au ticket.

```
pg_dump -Fc -d catalog_db -f catalog.dump; createdb restore_test; pg_restore -d restore_test catalog.dump
```

Controles de sortie

HTTP health vert; aucune augmentation des 5xx; latence dans le budget; aucune ligne cross-tenant; outbox sans age anormal; consumer lag stable; audit sans secret; rollback disponible.

Test a automatiser

Ajouter un test nomme `restauration_postgresql_regression` couvrant le succes, l'echec, la reprise et l'isolation du tenant demo. En preproduction, archiver resultat, version d'image et horodatage.

Runbook - Rollback applicatif (Preproduction)

Objectif

Revenir a l'image precedente uniquement si schema backward-compatible; ne jamais down-migrer automatiquement des donnees.

ProcEDURE exacte

1. Declarer l'incident ou le changement, le responsable, le tenant affecte et la fenetre. 2. Capturer les metriques avant action. 3. Executer d'abord en lecture seule. 4. Appliquer une seule modification reversible. 5. Verifier le walking skeleton complet. 6. Joindre preuves et decision au ticket.

```
kubect1 -n b2b rollout undo deployment/catalog-service
```

Controles de sortie

HTTP health vert; aucune augmentation des 5xx; latence dans le budget; aucune ligne cross-tenant; outbox sans age anormal; consumer lag stable; audit sans secret; rollback disponible.

Test a automatiser

Ajouter un test nomme rollback_applicatif_regression couvrant le succes, l'echec, la reprise et l'isolation du tenant demo. En preproduction, archiver resultat, version d'image et horodatage.

Runbook - Rotation secrets (Preproduction)

Objectif

Creer nouvelle version, deployer les consommateurs, verifier, puis revoquer l'ancienne sans imprimer la valeur.

ProcEDURE EXACTE

1. Declarer l'incident ou le changement, le responsable, le tenant affecte et la fenetre. 2. Capturer les metriques avant action. 3. Executer d'abord en lecture seule. 4. Appliquer une seule modification reversible. 5. Verifier le walking skeleton complet. 6. Joindre preuves et decision au ticket.

```
kubect1 -n b2b rollout restart deployment/gateway
```

Controles de sortie

HTTP health vert; aucune augmentation des 5xx; latence dans le budget; aucune ligne cross-tenant; outbox sans age anormal; consumer lag stable; audit sans secret; rollback disponible.

Test a automatiser

Ajouter un test nomme rotation_secrets_regression couvrant le succes, l'echec, la reprise et l'isolation du tenant demo. En preproduction, archiver resultat, version d'image et horodatage.

Runbook - SLO API (Preproduction)

Objectif

Mesurer disponibilite, latence p95/p99 et taux d'erreur par route/tenant sans cardinalite non bornee.

Procedure exacte

1. Declarer l'incident ou le changement, le responsable, le tenant affecte et la fenetre. 2. Capturer les metriques avant action. 3. Executer d'abord en lecture seule. 4. Appliquer une seule modification reversible. 5. Verifier le walking skeleton complet. 6. Joindre preuves et decision au ticket.

```
kubect1 -n b2b top pods
```

Controles de sortie

HTTP health vert; aucune augmentation des 5xx; latence dans le budget; aucune ligne cross-tenant; outbox sans age anormal; consumer lag stable; audit sans secret; rollback disponible.

Test a automatiser

Ajouter un test nomme slo_api_regression couvrant le succes, l'echec, la reprise et l'isolation du tenant demo. En preproduction, archiver resultat, version d'image et horodatage.

Runbook - Test de charge (Preproduction)

Objectif

Injecter catalogue et commandes realistes; verifier pool JDBC, Kafka lag, CPU, memoire et contention SQL.

ProcEDURE exacte

1. Declarer l'incident ou le changement, le responsable, le tenant affecte et la fenetre. 2. Capturer les metriques avant action. 3. Executer d'abord en lecture seule. 4. Appliquer une seule modification reversible. 5. Verifier le walking skeleton complet. 6. Joindre preuves et decision au ticket.

```
k6 run smoke.js
```

Controles de sortie

HTTP health vert; aucune augmentation des 5xx; latence dans le budget; aucune ligne cross-tenant; outbox sans age anormal; consumer lag stable; audit sans secret; rollback disponible.

Test a automatiser

Ajouter un test nomme test_de_charge_regression couvrant le succes, l'echec, la reprise et l'isolation du tenant demo. En preproduction, archiver resultat, version d'image et horodatage.

Runbook - Incident isolation tenant (Preproduction)

Objectif

Stopper fuite, conserver preuves, identifier tenants/objets, corriger predicate et ajouter test de regression.

Procédure exacte

1. Declarer l'incident ou le changement, le responsable, le tenant affecte et la fenetre. 2. Capturer les metriques avant action. 3. Executer d'abord en lecture seule. 4. Appliquer une seule modification reversible. 5. Verifier le walking skeleton complet. 6. Joindre preuves et decision au ticket.

```
select tenant_id,count(*) from products group by tenant_id;
```

Controles de sortie

HTTP health vert; aucune augmentation des 5xx; latence dans le budget; aucune ligne cross-tenant; outbox sans age anormal; consumer lag stable; audit sans secret; rollback disponible.

Test a automatiser

Ajouter un test nomme incident_isolation_tenant_regression couvrant le succes, l'echec, la reprise et l'isolation du tenant demo. En preproduction, archiver resultat, version d'image et horodatage.

Runbook - Validation Kubernetes (Preproduction)

Objectif

Valider schema, policy, images non-root, resources, probes et absence de secret en clair.

Procédure exacte

1. Declarer l'incident ou le changement, le responsable, le tenant affecte et la fenetre. 2. Capturer les metriques avant action. 3. Executer d'abord en lecture seule. 4. Appliquer une seule modification reversible. 5. Verifier le walking skeleton complet. 6. Joindre preuves et decision au ticket.

```
kubect1 apply --dry-run=server -f infra/k8s/
```

Controles de sortie

HTTP health vert; aucune augmentation des 5xx; latence dans le budget; aucune ligne cross-tenant; outbox sans age anormal; consumer lag stable; audit sans secret; rollback disponible.

Test a automatiser

Ajouter un test nomme validation_kubernetes_regression couvrant le succes, l'echec, la reprise et l'isolation du tenant demo. En preproduction, archiver resultat, version d'image et horodatage.

Runbook - Build reproductible (Preproduction)

Objectif

Utiliser lockfiles, versions d'images par digest, Maven dependency tree et SBOM archivee.

Procédure exacte

1. Declarer l'incident ou le changement, le responsable, le tenant affecte et la fenetre. 2. Capturer les metriques avant action. 3. Executer d'abord en lecture seule. 4. Appliquer une seule modification reversible. 5. Verifier le walking skeleton complet. 6. Joindre preuves et decision au ticket.

```
mvn -DskipTests dependency:tree; npm --prefix web ci
```

Controles de sortie

HTTP health vert; aucune augmentation des 5xx; latence dans le budget; aucune ligne cross-tenant; outbox sans age anormal; consumer lag stable; audit sans secret; rollback disponible.

Test a automatiser

Ajouter un test nomme build_reproductible_regression couvrant le succes, l'echec, la reprise et l'isolation du tenant demo. En preproduction, archiver resultat, version d'image et horodatage.

Runbook - Release canary (Preproduction)

Objectif

Deployer faible trafic, comparer erreurs/latence/Kafka lag, promouvoir ou rollback selon seuil predefini.

ProcEDURE EXACTE

1. Declarer l'incident ou le changement, le responsable, le tenant affecte et la fenetre. 2. Capturer les metriques avant action. 3. Executer d'abord en lecture seule. 4. Appliquer une seule modification reversible. 5. Verifier le walking skeleton complet. 6. Joindre preuves et decision au ticket.

```
kubect1 -n b2b rollout status deployment/order-service
```

Controles de sortie

HTTP health vert; aucune augmentation des 5xx; latence dans le budget; aucune ligne cross-tenant; outbox sans age anormal; consumer lag stable; audit sans secret; rollback disponible.

Test a automatiser

Ajouter un test nomme release_canary_regression couvrant le succes, l'echec, la reprise et l'isolation du tenant demo. En preproduction, archiver resultat, version d'image et horodatage.

Runbook - Observabilite trace (Preproduction)

Objectif

Propager traceparent et correlation-id HTTP vers messages Kafka; relier commande et evenement sans PII.

ProcEDURE EXACTE

1. Declarer l'incident ou le changement, le responsable, le tenant affecte et la fenetre. 2. Capturer les metriques avant action. 3. Executer d'abord en lecture seule. 4. Appliquer une seule modification reversible. 5. Verifier le walking skeleton complet. 6. Joindre preuves et decision au ticket.

```
curl -H 'traceparent: 00-00000000000000000000000000000001-0000000000000001-01' ...
```

Controles de sortie

HTTP health vert; aucune augmentation des 5xx; latence dans le budget; aucune ligne cross-tenant; outbox sans age anormal; consumer lag stable; audit sans secret; rollback disponible.

Test a automatiser

Ajouter un test nomme observabilite_trace_regression couvrant le succes, l'echec, la reprise et l'isolation du tenant demo. En preproduction, archiver resultat, version d'image et horodatage.

Runbook - Nettoyage outbox (Preproduction)

Objectif

Supprimer par lots les evenements publies au-dela de retention, apres verification audit et replay.

ProcEDURE EXACTE

1. Declarer l'incident ou le changement, le responsable, le tenant affecte et la fenetre. 2. Capturer les metriques avant action. 3. Executer d'abord en lecture seule. 4. Appliquer une seule modification reversible. 5. Verifier le walking skeleton complet. 6. Joindre preuves et decision au ticket.

```
delete from outbox_events where id in (select id from outbox_events where published_at < now()-interval '30 days'
limit 1000);
```

Controles de sortie

HTTP health vert; aucune augmentation des 5xx; latence dans le budget; aucune ligne cross-tenant; outbox sans age anormal; consumer lag stable; audit sans secret; rollback disponible.

Test a automatiser

Ajouter un test nomme nettoyage_outbox_regression couvrant le succes, l'echec, la reprise et l'isolation du tenant demo. En preproduction, archiver resultat, version d'image et horodatage.

Runbook - DR exercice (Preproduction)

Objectif

Simuler perte region/cluster, restaurer donnees, DNS et cles; documenter temps reel et ecarts.

ProcEDURE exacte

1. Declarer l'incident ou le changement, le responsable, le tenant affecte et la fenetre. 2. Capturer les metriques avant action. 3. Executer d'abord en lecture seule. 4. Appliquer une seule modification reversible. 5. Verifier le walking skeleton complet. 6. Joindre preuves et decision au ticket.

```
kind delete cluster --name b2b-mvp; ./scripts/kind-up.ps1
```

Controles de sortie

HTTP health vert; aucune augmentation des 5xx; latence dans le budget; aucune ligne cross-tenant; outbox sans age anormal; consumer lag stable; audit sans secret; rollback disponible.

Test a automatiser

Ajouter un test nomme dr_exercice_regression couvrant le succes, l'echec, la reprise et l'isolation du tenant demo. En preproduction, archiver resultat, version d'image et horodatage.

Runbook - Revue avant production (Preproduction)

Objectif

Supprimer comptes demo, mots de passe locaux et start-dev; activer TLS, HA, sauvegardes, politiques et alertes.

Procédure exacte

1. Déclarer l'incident ou le changement, le responsable, le tenant affecté et la fenêtre. 2. Capturer les métriques avant action. 3. Exécuter d'abord en lecture seule. 4. Appliquer une seule modification réversible. 5. Vérifier le walking skeleton complet. 6. Joindre preuves et décision au ticket.

```
rg 'local-only-change-me|start-dev|buyer@demo' .
```

Contrôles de sortie

HTTP health vert; aucune augmentation des 5xx; latence dans le budget; aucune ligne cross-tenant; outbox sans âge anormal; consumer lag stable; audit sans secret; rollback disponible.

Test à automatiser

Ajouter un test nommé `revue_avant_production_regression` couvrant le succès, l'échec, la reprise et l'isolation du tenant demo. En préproduction, archiver résultat, version d'image et horodatage.

Elements CLI/boilerplate volontairement non listes

Les fichiers metier requis sont listes integralement. Les elements suivants sont generes par les CLI standard et ne doivent pas etre recopies a la main: Maven Wrapper (.mvn, mvnw, mvnw.cmd), package-lock.json produit par npm ci/install puis commite, fichiers Angular Karma eventuellement generes, icones et assets, dossiers Flutter android/ios/web/windows/linux/macos, Gradle wrapper, Podfile, fichiers IDE. Commandes de generation:

```
mvn wrapper:wrapper
npx @angular/cli@22 new web --standalone --routing --style=css --skip-git
flutter create --platforms=android,ios,web mobile
# Puis remplacer uniquement les fichiers explicitement fournis dans ce guide.
```

Limites honnetes du walking skeleton

Le PKCE Angular illustre le point d'entree mais doit etre confie a angular-auth-oidc-client avec callback, state, nonce et code_verifier complets. Le provisionnement des bases init.sql est local; Kubernetes exige un job/operator. Le stock est decremente de facon eventuelle sans reservation ni idempotence persistante. Kafka et Keycloak mono-replica conviennent a kind, pas a la production. Les manifests sont une base cloud-agnostique, a completer par NetworkPolicy, HPA, PDB, TLS/cert-manager, External Secrets et stockage adapte.

Definition of Done MVP

Un utilisateur demo s'authentifie; le JWT contient tenant_id; il cree/liste un produit, initialise/liste son stock, cree/liste une commande; la commande et l'outbox sont atomiques; orders.created.v1 est publie puis consomme; un autre tenant ne voit aucune donnee; Maven/Angular/Flutter passent; Compose demarre; kind applique les manifests; aucun secret reel n'est commite.